OpenS2P
Architecture Vision
v4.4 — Final Enterprise Reference Standard

Open Operating System for Procurement AI

Version	4.4
Status	Approved — Final Architecture Reference Standard
Document Class	Authoritative Reference Architecture
Supersedes	OpenS2P Architecture Vision v3.0 – v4.3
Effective Date	Q1 2027
Audience	Contributors · Architects · Enterprise Adopters · Security Reviewers
Maintained by	OpenS2P Core Team
License	Apache 2.0

This document is the single authoritative technical reference for OpenS2P platform architecture, design decisions, extension model, security posture, deployment topology, and governance model.

Table of Contents

How to Read This Document

OpenS2P Architecture Vision v4.4 is a multi-audience reference document. Different readers have different entry points. Use this guide to navigate efficiently.

If you are a...	Start here	Key chapters	Skip
New contributor	§1 Executive Summary → §1.2 Goals → §1.3 Constraints → §3.1 Domain Ownership	Parts I–III, ADR Catalog §23	Deployment topology details
Platform architect	§1.4 Traceability Matrix → Part II C4 → Part III Domain	Parts I–IV, §18 Security, §23 ADRs	§20 Testing internals
Enterprise adopter	§1.2 Goals → §19 Deployment → §21 Compliance	§19 Topology, §21 Compliance, §24 Radar	Part IV AI internals
Security reviewer	§18 Security Reference → §1.3 Constraints → §21 Compliance	§18, §21, §13 AI Eval, ADR-008/009	Plugin lifecycle internals
AI engineer	Part IV AI Architecture → §11 Prompt → §12 Memory → §13 Evaluation	Parts IV–V, ADR-003/010/016	Non-AI domain chapters
Plugin/connector developer	§16 Plugin Runtime → §17 Marketplace → §23 ADR-006–007	Parts V–VI, §23 ADR-006–011	Compliance and DR sections
SRE / infrastructure	§19 Deployment Profiles → §19.4 Capacity → §19.5 DR → §20 Testing	§19, §20 Testing, Observability	Part IV AI internals
Compliance officer	§21 Compliance → §13.1 Eval Metrics → §18 Security	§21, §18, §13, §9.1 Lineage	Plugin and connector chapters

Navigation Guide: Entry Points by Reader Role

📖	**RFC 2119 Language Used Throughout** This document uses MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY with the meanings defined in RFC 2119. MUST = absolute requirement. SHOULD = strong recommendation with legitimate exceptions. MAY = optional feature.

PART I

POSITIONING AND SCOPE

1. Executive Summary & Strategic Positioning

🎯	**The Architectural Constitution** This document is not a design proposal — it is the architectural constitution of the OpenS2P platform. It defines the stable system model, key abstractions, deployment expectations, security reference patterns, extension model, governance flow, and decision history that contributors and adopters can rely on over time.

OpenS2P is an open-source, AI-native operating platform for enterprise Source-to-Pay procurement. It functions not as a monolithic application but as a modular, extensible platform on which procurement intelligence, workflow automation, supplier management, contract lifecycle, and financial controls are expressed as composable first-class capabilities.

This document — OpenS2P Architecture Vision v4.4 — supersedes all prior architecture drafts. It expands v4.1 with a formal Business Capability Model, Domain Dependency Map, consolidated AI Architecture, Prompt Architecture standard, Memory Architecture, AI Evaluation Framework, Testing Architecture, and Compliance Architecture.

1.1 Platform Purpose

Enterprise procurement remains one of the least automated, most fragmented areas of business operations. Organisations spend 70–85% of their procurement cycles on low-value data entry, status chasing, and manual exception handling. OpenS2P proposes a different architecture: a vendor-neutral, API-first, AI-augmented procurement platform where intelligence is embedded at every decision gate and every capability can be extended or replaced by the community.

2. Business Capability Model

The Business Capability Model is a standard enterprise architecture artifact that maps what the platform does — independently of how it is implemented. Every architecture decision, feature proposal, and roadmap item maps to one or more capabilities in this hierarchy.

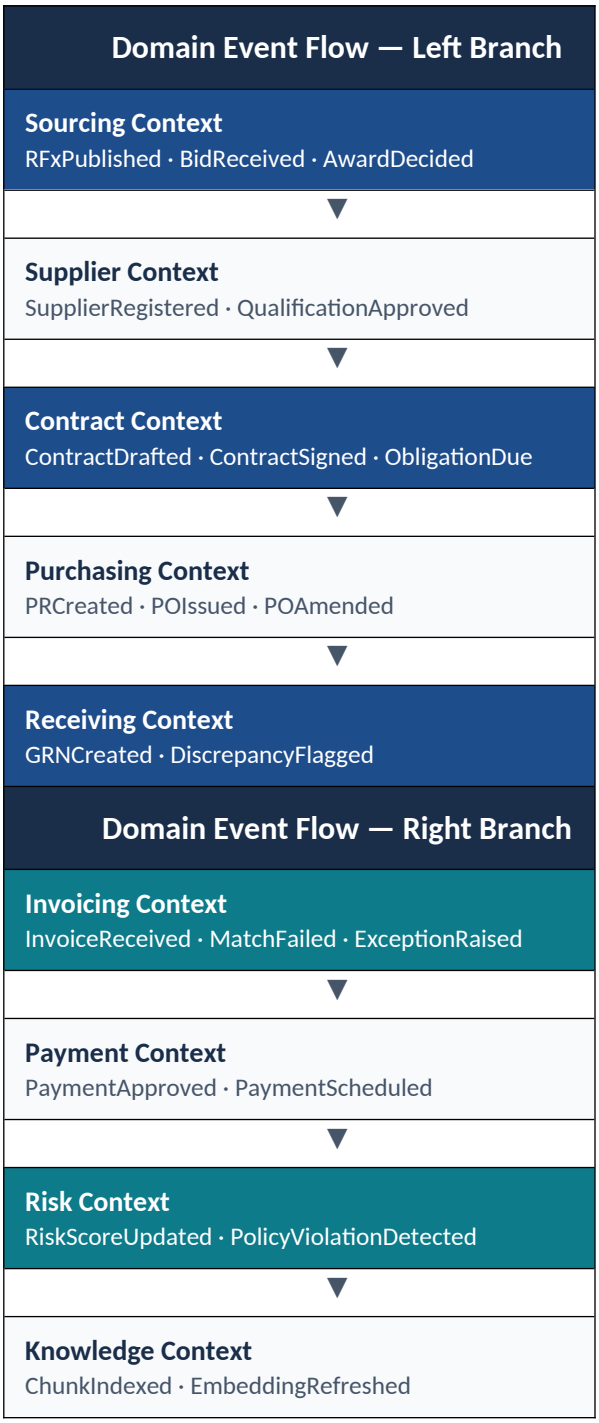
OpenS2P Business Capability Hierarchy		
Strategic Sourcing	• RFx & Tendering	• Supplier Discovery
	• Bid Analysis & Award	• Auction Management
	• Category Strategy	• Savings Tracking

Supplier Management	• Supplier Registration	• Qualification & Vetting
	• Performance Scoring	• Financial Health Monitor
	• Diversity & ESG Tracking	• Supplier Portal
Contract Lifecycle	• AI Contract Generation	• Contract Review & Redline
	• Obligation Management	• Renewal Management
	• Risk Scoring	• CLM Analytics
Procurement Operations	• Purchase Requisition	• Purchase Order Lifecycle
	• Goods Receipt / GRN	• Three-Way Matching
	• Catalogue Management	• Tail Spend Control
Invoice Management	• Invoice Capture (OCR/EDI)	• Exception Management
	• Early Payment Programs	• Supplier Self-Service
	• AP Automation	• Invoice Analytics
Spend Analytics	• Spend Classification (AI)	• Category Analytics
	• Maverick Spend Detection	• Savings Realisation
	• Budget vs Actual	• ESG Spend Reporting
Risk & Compliance	• Policy-as-Code Enforcement	• Regulatory Compliance
	• Sanctions & AML Checks	• SOX Controls
	• Audit Evidence Packs	• Risk Dashboard
AI & Automation	• Agent Orchestration	• Autonomous Approvals
	• Anomaly Detection	• Document Intelligence
	• Knowledge Graph Q&A	• Confidence Governance
Platform & Ecosystem	• Plugin Marketplace	• Connector Framework
	• SDK & APIs	• Identity & Access
	• Observability	• Governance & Audit

Figure 1: OpenS2P Business Capability Hierarchy

3. Domain Dependency Map

The Domain Dependency Map shows how the ten bounded contexts relate to each other through event-driven dependencies. The arrows represent the primary event flow: upstream domains produce events that downstream domains consume. No downstream domain may call upstream APIs synchronously — all integration is via Kafka events.



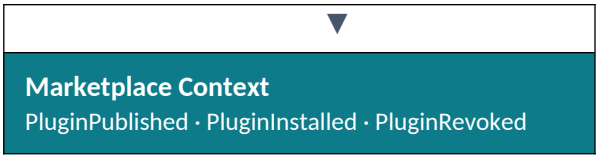


Figure 2: Domain Dependency Map — Event Flow Between Bounded Contexts

3.1 Domain Ownership Matrix

Every bounded context has a designated owning team. Domain ownership is the organisational coupling of a team to a bounded context. The owning team has final decision-making authority over the domain model, API contracts, event schemas, and ADRs within their context. Shared ownership is not permitted — shared ownership means no ownership.

Bounded Context	Owning Team	Core Responsibilities	Primary Interfaces	On-Call
Sourcing	Sourcing Team	RFx lifecycle, bid management, award decisions, savings tracking	REST API · Kafka events · Sourcing Agent	Sourcing Lead
Supplier	Supplier Team	Supplier onboarding, qualification, performance, financial health	REST API · Kafka events · Supplier Agent · Connector (D&B, Creditsafe)	Supplier Lead
Contract	Contract Team	CLM lifecycle, obligation tracking, risk scoring, renewal management	REST API · Kafka events · Contract Analysis Agent · CLM AI pipeline	Contract Lead
Purchasing	Procurement Team	PR/PO lifecycle, catalogue, approval routing, three-way match prep	REST API · Kafka events · ERP Connector	Procurement Lead
Receiving	Procurement Team	GRN creation, discrepancy flagging, receipt confirmation	REST API · Kafka events	Procurement Lead
Invoicing	AP Team	Invoice capture, three-way match, exception management, early payment	REST API · Kafka events · Invoice Agent · OCR Connector	AP Lead

Payment	AP Team	Payment scheduling, approval, ERP writeback	REST API · Kafka events · Payment Rail Connector	AP Lead
Risk	Risk & Compliance Team	Risk profiling, signal aggregation, policy violation detection	REST API · Kafka events · Risk Agent · External Risk Connectors	Risk Lead
Knowledge	AI Platform Team	Document embedding, vector index, organisational memory pipeline	Internal gRPC · Kafka events · Embedding Pipeline	AI Platform Lead
Marketplace	Platform Team	Plugin publication, trust services, distribution, tenant installation	REST API · Kafka events · Trust Service · CDN	Platform Lead

Figure E: Domain Ownership Matrix

3.2 Ownership Rules

- A domain owner MUST review and approve any PR that modifies the domain's OpenAPI spec, AsyncAPI schema, aggregate root, or Kafka topic definition.
- Cross-domain integration changes require approval from BOTH affected domain owners.
- A domain owner MAY delegate day-to-day review to module owners within their team, but retains architectural accountability.
- Domain ownership transfers require a TSC decision recorded in the ADR catalog.
- The on-call rotation for a domain is the responsibility of the owning team. Platform Team SREs provide only cross-cutting infrastructure on-call support.

3.3 DDD Context Map

The DDD Context Map defines the structural relationships between bounded contexts using Domain-Driven Design terminology. It goes beyond the event flow diagram by describing the nature of each relationship: who is upstream, who is downstream, and what translation or anti-corruption mechanisms exist at each boundary.

ℹ️	**DDD Relationship Terms** Upstream (U): Context that publishes and defines the model. Downstream (D): Context that consumes and adapts. Published Language (PL): Well-defined versioned event schema. Anti-Corruption Layer

	(ACL): Translation layer shielding downstream from upstream model changes. Open Host Service (OHS): Standardised API for any consumer. Conformist (CF): Downstream adopts upstream model as-is.

Upstream (U)	Relationship	Downstream (D)	DDD Pattern	Boundary Mechanism
Sourcing	U → D	Supplier	Published Language	Kafka: RFxPublished. Supplier reads supplier IDs from sourcing events. Supplier model not exposed upstream.
Supplier	U → D	Contract	PL + ACL	Kafka: SupplierRegistered, QualificationApproved. Contract ACL translates supplier profile into ContractParty model.
Supplier	U → D	Risk	Published Language	Kafka: SupplierRegistered, RiskFlagRaised. Risk maintains its own RiskProfile — does not share Supplier aggregate.
Contract	U → D	Purchasing	PL + ACL	Kafka: ContractSigned. Purchasing reads framework terms via ACL. ACL translates Contract terms into PO terms model.
Purchasing	U → D	Receiving	Conformist	Kafka: POIssued. Receiving adopts PO line structure directly. Tight coupling accepted — same team owns both contexts.
Purchasing	U → D	Invoicing	Published Language	Kafka: POIssued, POAmended.

				Invoicing reads PO lines via ACL for three-way match. Invoice model independent.
Receiving	U → D	Invoicing	Published Language	Kafka: GRNCreated. Invoicing reads GRN quantities for match engine. GRN model translated at ACL boundary.
Invoicing	U → D	Payment	Open Host Service	Payment calls Invoice OHS REST API for approved amounts. Kafka: InvoiceApproved also published.
Risk	U → D (cross-cutting)	Contract + Sourcing + Invoicing	Published Language	Kafka: RiskScoreUpdated, PolicyViolationDetected. All downstream subscribe but translate signals independently.
Knowledge	U → D (supporting)	All AI-enabled contexts	Open Host Service	gRPC: Vector search API and graph query API as OHS. Agents use these without coupling to Knowledge aggregate.
Marketplace	U → D (platform)	All plugin consumers	OHS + ACL	REST: Plugin capability calls via OHS. Each context uses ACL to translate plugin outputs to own model.
External ERP (SAP/Oracle)	U (legacy) → D	Purchasing, Supplier	Anti-Corruption Layer	Connector Framework provides full ACL. ERP data models (BAPI/IDoc) translated to OpenS2P canonical models.

Figure P: DDD Context Map — Bounded Context Relationships

3.4 Context Map Rules

- ACL MUST be implemented whenever a downstream context integrates with an upstream context whose model may change independently. The ACL isolates change at the boundary.
- Published Language event schemas MUST be documented in AsyncAPI before the first event is published. Schema changes require the dual-publish compatibility period (ADR-014).
- Open Host Services MUST expose OpenAPI 3.1 specifications. OHS contracts are platform-public stable interfaces.
- Conformist relationships MUST be justified in an ADR comment when both contexts share an owning team. If team ownership diverges, Conformist MUST be refactored to Published Language.
- External system integrations MUST always use ACL via the Connector Framework. No external data model may appear directly in a core domain aggregate.

3.5 Cross-Context Event Integration Rules

- Upstream domains MUST publish events to named Kafka topics before any downstream domain can consume them. (Constraint C-05)
- Downstream domains MUST NOT call upstream domain REST APIs for data that can be obtained via events.
- Circular event dependencies are prohibited — any proposal that creates a cycle requires TSC review.
- Every event has a canonical AsyncAPI specification. Schema evolution follows the dual-publish compatibility policy (ADR-014).

1.2 Architecture Goals

Architecture Goals define the north star for every design decision in OpenS2P. They answer WHY the architecture is shaped the way it is. Every ADR, principle, and technology choice is evaluated first against this list. If a proposal contradicts a goal, it requires explicit TSC approval and a documented trade-off.

#	Goal	What It Means in Practice	Primary Realization
G-01	Cloud Native	Platform-first design for containerised, orchestrated runtimes. Stateless services, health probes, graceful shutdown, immutable images.	Kubernetes · Docker · Helm
G-02	AI Native	AI is not a feature layer — it is an architectural primitive. Agents, confidence scoring, and HITL gates are first-class platform concepts.	LangGraph · vLLM · PGVector · Prompt Registry
G-03	API First	Every capability is an API before it is a UI. Internal and external consumers are indistinguishable at the contract level.	OpenAPI 3.1 · AsyncAPI 3.x · Contract-first CI gate
G-04	Event Driven	Domain boundaries crossed exclusively via immutable, typed, versioned events. Synchronous cross-domain calls are an anti-pattern.	Kafka · AsyncAPI · Domain Event Catalog
G-05	Enterprise Secure	Zero-trust posture end to end. No implicit trust. Policy-as-code. Dynamic secrets. Immutable audit. PII-safe AI pipelines.	OPA · Keycloak · Vault · ClickHouse · PII Redactor
G-06	Modular	Every capability is independently deployable, replaceable, and testable. The platform has no 'God service'.	Bounded Contexts · Separate Deployables · Connector Framework
G-07	Extensible	Enterprise adopters and	Plugin Marketplace ·

		ISVs can extend the platform without modifying core services. Extension points are stable, documented, and compatibility-guaranteed.	Connector Framework · Webhook Dispatcher · SDK
G-08	Vendor Neutral	No strategic lock-in to any single cloud provider, LLM provider, ERP vendor, or infrastructure toolchain. Swap-able at every layer.	Polyglot Persistence · LLM Abstraction Layer · Cloud-Agnostic Topology
G-09	Multi Tenant	Multi-tenancy is a first-class architectural concern built into the data layer, identity layer, and policy layer from day one — not retrofitted.	Schema-per-tenant · OPA Tenant Claims · Keycloak Realms
G-10	Open Source Friendly	Low contributor barrier. Conventional commits, clear module ownership, comprehensive testing, permissive license, and contributor documentation.	Apache 2.0 · ADR Governance · RFC Process · Domain Ownership Matrix

Figure A: Architecture Goals — OpenS2P North Star

1.3 Architecture Constraints

Architecture Constraints are non-negotiable boundaries within which the architecture MUST operate. Unlike principles (which guide decisions) and goals (which describe intent), constraints are hard limits. A proposal that violates a constraint is rejected outright — no trade-off discussion is available unless a constraint is itself formally revised through the ADR process.

Architecture Constraints — Hard Limits
C-01 MUST run on Kubernetes 1.28+. No bare-metal or VM-only deployment target is supported as a primary path. Docker Compose is supported for developer environments only.
C-02 MUST support both SaaS multi-tenant and self-hosted enterprise (air-gapped) deployment without code changes. Configuration determines topology.
C-03 MUST function without external AI APIs. Every AI capability MUST have a self-hosted inference path

(vLLM + open-weight models). External LLM APIs are an optional optimisation, not a requirement.
C-04 PostgreSQL 15+ is the mandatory default operational store. Deviation requires an ADR and TSC approval. No MySQL, SQLite, or proprietary RDBMS as a substitute.
C-05 Apache Kafka is the mandatory event streaming backbone. All cross-domain integration MUST use Kafka topics. In-process event buses are permitted only within a single bounded context.
C-06 Every service MUST publish structured domain events to Kafka for every state transition of a system-of-record entity. Silent state changes are a bug.
C-07 Every REST API MUST be documented in an OpenAPI 3.1 specification committed before implementation. The spec is the source of truth.
C-08 Every event schema MUST be documented in an AsyncAPI 3.x specification. Event schema changes require a dual-publish compatibility period (ADR-014).
C-09 Multi-tenancy is mandatory. No single-tenant-only design decisions are accepted. Every new feature must specify its multi-tenancy behaviour.
C-10 No shared database schema between bounded contexts. Each bounded context owns exactly one primary PostgreSQL schema. Cross-context data access is via events or read-model projections only.
C-11 All services MUST instrument with OpenTelemetry SDK. Proprietary vendor observability SDKs are not permitted in platform code (ADR-012).
C-12 All AI agent prompts MUST pass through the PII Redactor before transmission to any LLM — self-hosted or external. No PII in any LLM prompt is an absolute constraint.

Figure B: Architecture Constraints — Non-Negotiable Hard Limits

1.4 Architecture Principle Traceability Matrix

Every architecture principle (P1–P7) is realised through specific platform components, standards, and controls. This traceability matrix makes principles measurable — each principle has a concrete implementation that can be audited, tested, and enforced in CI.

Principle	Statement (short)	Platform Realization	Enforcement Mechanism	ADR Reference
P1 — API First	Every capability is an API before a UI	OpenAPI 3.1 specs committed before code; AsyncAPI for events	CI contract test gate rejects divergence	ADR-013
P2 — Event Driven	Cross-domain integration via	Kafka topics; AsyncAPI catalog;	Architecture review rejects synchronous	ADR-002

	events only	domain event taxonomy	cross-domain REST	
P3 — Zero Trust	No implicit trust between services	OPA policy engine; Keycloak JWT; Vault dynamic secrets; mTLS	Security gate in CI; OPA decision logging; Vault audit	ADR-008, ADR-009
P4 — Observable	OTEL instrumentation from day one	OpenTelemetry SDK mandatory; OTEL Collector in all topologies	CI rejects services without OTEL instrumentation	ADR-012
P5 — Extension Without Modification	Core services are not modified to add capabilities	Plugin Marketplace; Connector Framework; Webhook Dispatcher; SDK	Plugin Capability Guard via OPA; manifest required	ADR-006, ADR-007
P6 — Human Authority	Agents cannot bypass approval gates	Confidence Scorer + Approval Gate Service; HITL gate in LangGraph	Workflow graph enforces gate; bypass attempt is logged and blocked	ADR-003, ADR-016
P7 — Measurable AI Quality	AI quality is a platform SLO	AI Evaluation Framework; Prompt Registry; ClickHouse AI audit	CI gate blocks prompt deployment without eval pass	ADR-016

Figure C: Architecture Principle Traceability Matrix

1.5 Architecture Decision Summary — The "Why" Reference

This table provides a rapid-reference answer to the most common question contributors and architects ask: "Why did you choose X instead of Y?" Full rationale is in the ADR Catalog (Chapter 23). This summary is for onboarding and review contexts.

Technology Choice	Why Chosen	Primary Alternative Rejected	ADR
Kafka (not RabbitMQ)	Durable, ordered, replayable log. Kafka's consumer group model enables independent domain scaling. Essential for audit trail replay.	RabbitMQ — no durable ordered replay; message loss under failure	ADR-002
PostgreSQL (not	ACID guarantees, row-	MongoDB — flexible	ADR-004

MongoDB)	level security, schema-per-tenant, PGVector co-location. Procurement data requires transactional consistency.	schema but weaker isolation and no native vector search	
PGVector (not Pinecone)	Co-located with PostgreSQL — no separate operational overhead. ACID semantics extend to vector search. Air-gap friendly.	Pinecone — managed SaaS only; not viable for air-gapped enterprise	ADR-004
Neo4j (not pure SQL graph)	Cypher query language for relationship traversal. Supplier networks and obligation webs require multi-hop graph queries that are impractical in SQL.	PostgreSQL recursive CTEs — possible but orders-of-magnitude slower for 4+ hop traversals	ADR-004
ClickHouse (not PostgreSQL audit tables)	Columnar insert-only analytics. PostgreSQL audit tables are mutable (DBA can delete). ClickHouse provides tamper-evidence at database layer.	PostgreSQL — DBA can issue DELETE; not tamper-evident without external controls	ADR-009
LangGraph (not Temporal)	Graph-based state machine maps directly to procurement workflows. Built-in suspension/resumption for HITL gates. Lower operational overhead.	Temporal — strong durability but heavyweight cluster; learning curve high for OSS contributors	ADR-003
OPA (not custom RBAC)	Policy-as-code. Externally auditable, version-controlled, testable. ABAC supports multi-attribute decisions (role + spend band + category + time).	Custom RBAC middleware — full control but no external auditability and no policy-as-code CI integration	ADR-008
Keycloak (not Auth0/Okta)	Self-hostable. Supports SAML 2.0 + OIDC. Enterprise LDAP/AD federation. Required for air-gapped deployments. No per-MAU pricing.	Auth0/Okta — managed SaaS only; per-MAU cost prohibitive at enterprise scale; not air-gap viable	ADR-009

Wasm (not Docker containers)	Sub-millisecond sandbox startup. Hard resource limits enforced by runtime. Binary portable. No OCI image pull overhead per plugin invocation.	Container-only — universal compatibility but 200–800ms startup overhead at plugin invocation frequency	ADR-007
vLLM (not Ollama in production)	PagedAttention enables 10–30× higher GPU throughput than naive serving. Continuous batching. Required for 500+ concurrent agent executions.	Ollama — excellent developer UX but single-request serving; throughput insufficient for production agent load	ADR-010

Figure D: Architecture Decision Summary — The Why Reference

4. Architecture Principles & Quality Attributes

4.1 Core Architecture Principles

OpenS2P Binding Architecture Principles

P1 — API-First
Every capability is expressed as a versioned API before a UI. Internal and external consumers are treated identically.

P2 — Event-Driven Integration
Domain boundaries are crossed exclusively via typed, versioned, immutable events. Synchronous cross-domain calls require an ADR exception.

P3 — Zero-Trust Security
No implicit trust between services. Every request authenticated. Policy centralised. Secrets dynamically injected — never in environment variables.

P4 — Observable by Design
OpenTelemetry traces, metrics, and logs emitted from day one. Observability is a platform contract, not a deployment afterthought.

P5 — Extension Without Modification
Platform provides stable connectors, plugins, event hooks, policy adapters. Core surface is minimised; seam area is maximised.

P6 — Human Authority at Uncertainty Thresholds

Agents act autonomously below a configurable confidence threshold. Above threshold a human gate is mandatory. No agent may bypass approval.

P7 — Measurable AI Quality

Every AI agent exposes evaluation metrics. Hallucination rate, citation coverage, tool success rate, and workflow completion are platform-level SLOs.

Figure 3: OpenS2P Binding Architecture Principles

4.2 Non-Functional Requirements

Quality Attribute	Requirement	Target	ADR
Availability	SaaS platform uptime	99.9% monthly	ADR-012
AI Hallucination Rate	Factual accuracy of agent outputs	< 1%	ADR-016
Citation Coverage	Evidence backing for AI recommendations	100%	ADR-016
Tool Success Rate	Agent tool call completion rate	> 99%	ADR-016
Workflow Completion	End-to-end agent workflow success	> 95%	ADR-016
API Latency	p95 response time	< 400 ms	ADR-005
Audit Completeness	Domain event capture rate	100% — no gaps	ADR-010
Plugin Isolation	Memory blast radius	Wasm hard limit 256MB	ADR-007
RTO	Recovery time — SaaS	< 1 hour	ADR-012
RPO	Recovery point — transactional	< 5 minutes	ADR-012

PART II

C4 ARCHITECTURAL BACKBONE

5. C4 Level 1: System Context

The Level 1 diagram presents OpenS2P as a single system surrounded by external actors and integrated systems. Its purpose is to define the outer boundary and make all external dependencies explicit.

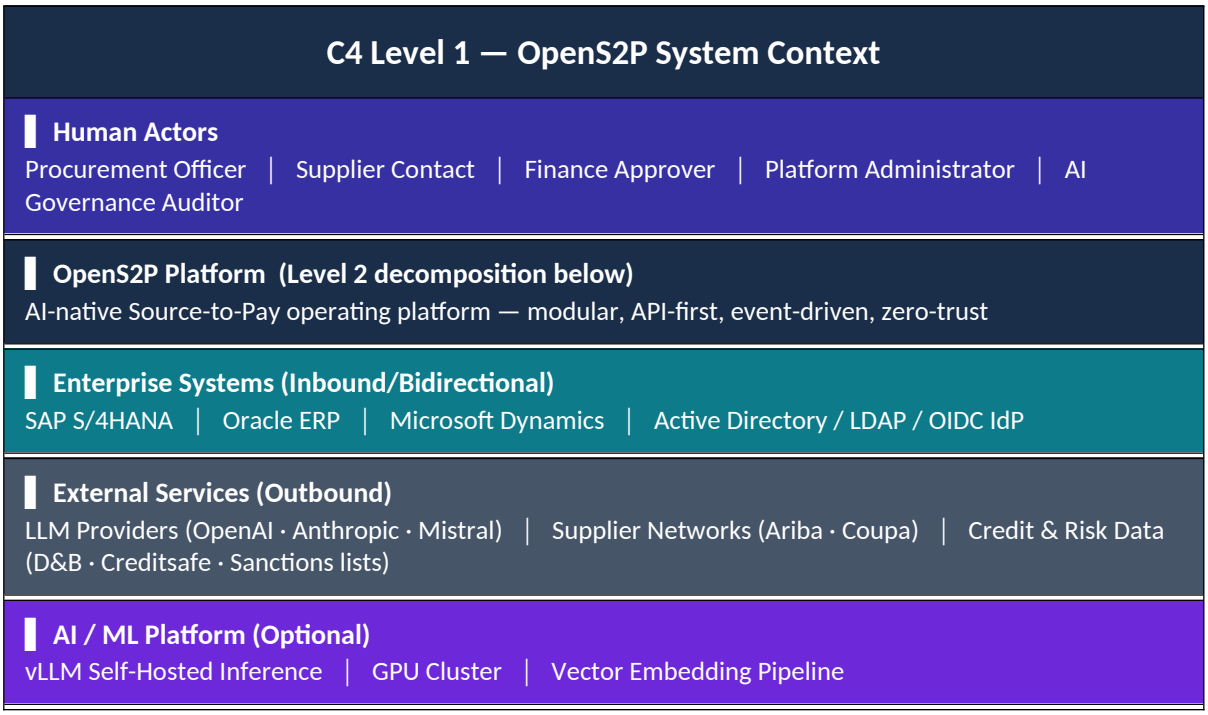
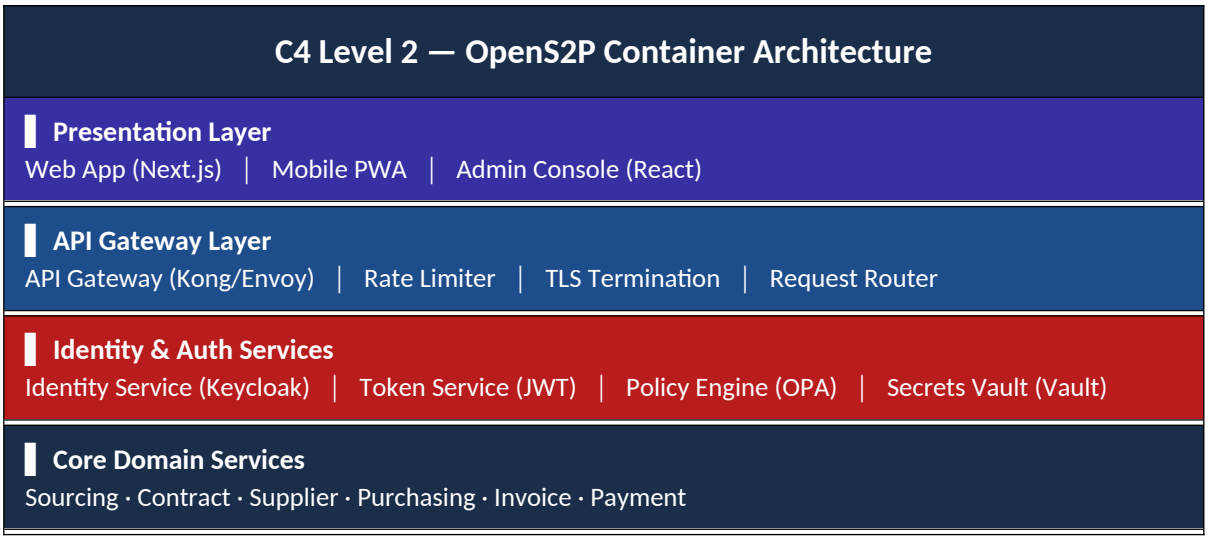


Figure 4: C4 Level 1 — System Context

6. C4 Level 2: Container Architecture



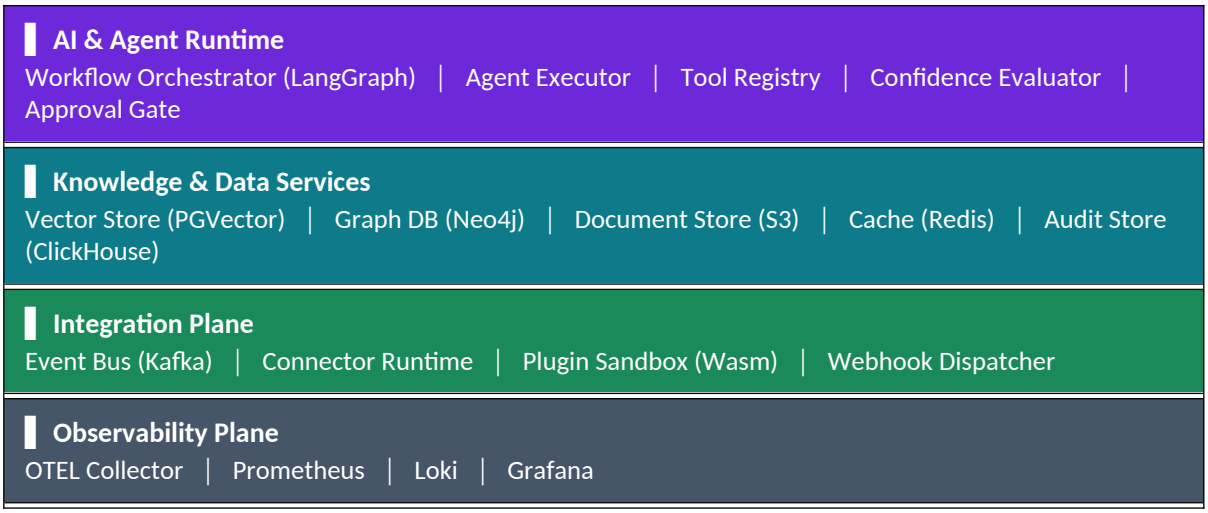


Figure 5: C4 Level 2 — Container Architecture

7. C4 Level 3: Component Architecture

7.1 Workflow Orchestration Components

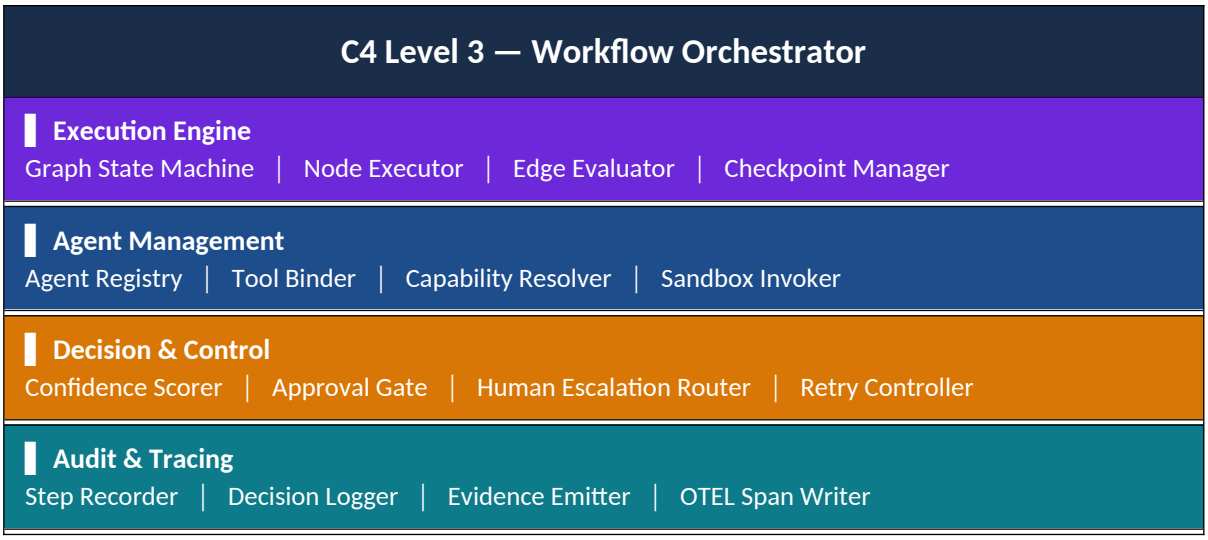


Figure 6: C4 Level 3 — Workflow Orchestration Components

7.2 Security Services Components

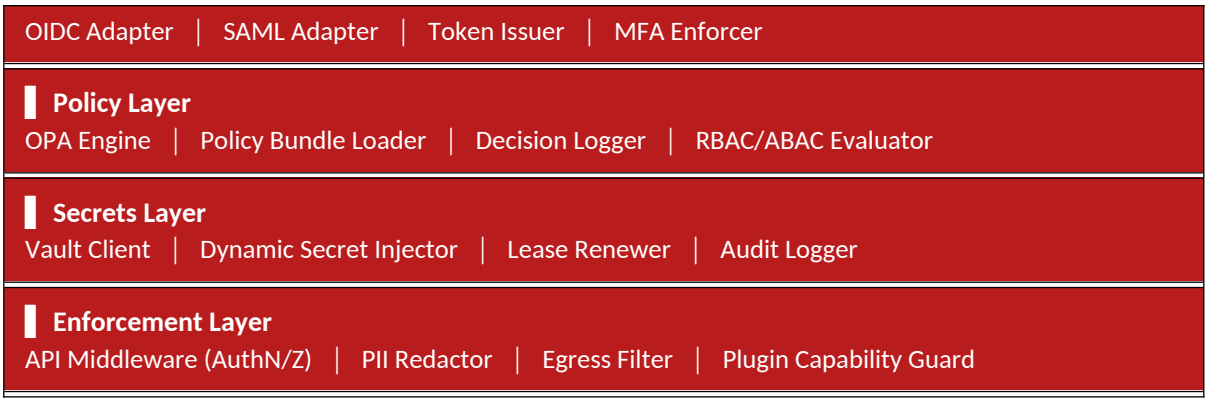


Figure 7: C4 Level 3 — Security Services Components

7.3 Marketplace Components

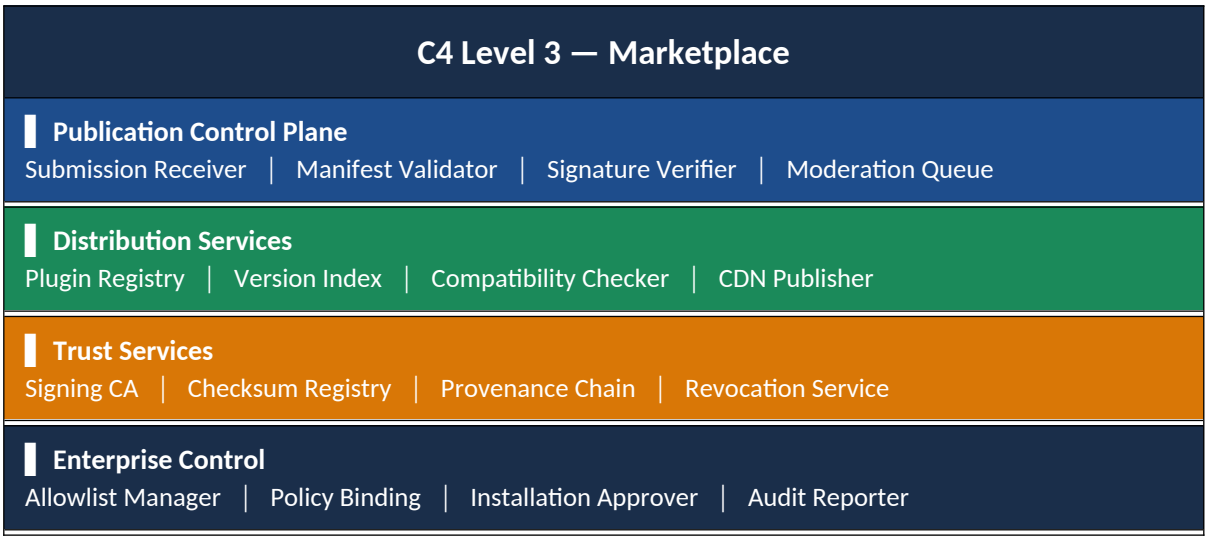
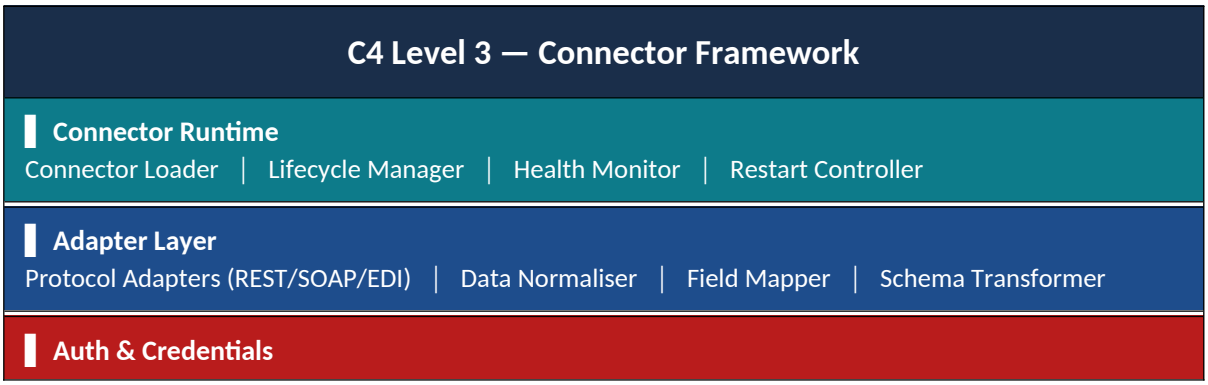


Figure 8: C4 Level 3 — Marketplace Components

7.4 Connector Framework Components



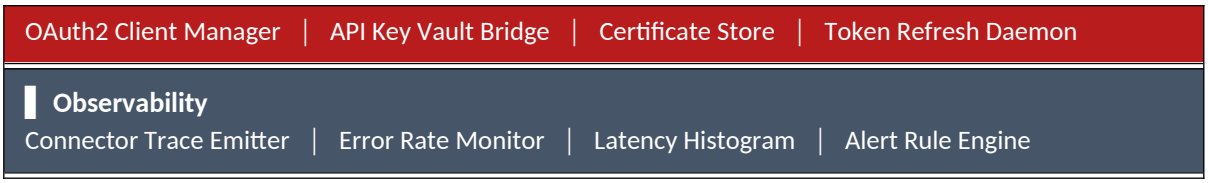


Figure 9: C4 Level 3 — Connector Framework Components

PART III

DOMAIN AND DATA ARCHITECTURE

8. Procurement Ontology & Bounded Contexts

The procurement ontology defines the canonical vocabulary of the OpenS2P domain. Every service, API, event, and agent prompt MUST use terms from this ontology. Synonyms from legacy systems are acceptable at integration boundaries but must be normalised inside the platform.

Core Procurement Ontology
Category — Spend taxonomy node. UNSPSC-aligned classification of goods and services.
Supplier — Legal entity providing goods or services. Has: legal name, registration IDs, addresses, certifications, financial profile.
Contract — Universal agreement aggregate spanning procurement, sales, NDA, MSA, SOW, employment variants.
Purchase Requisition — Internal demand document. States: requester, cost centre, line items, estimated value, required date.
Purchase Order — External commitment derived from an approved PR. Binds: supplier, delivery terms, payment terms.
Goods Receipt — Confirmation of physical or digital receipt against a PO line.
Invoice — Supplier's payment claim. Three-way match target: PO line + GRN quantity + invoice amount.
Payment — Financial settlement. ERP is system of record. OpenS2P holds approval state and schedule.
Obligation — Actionable commitment derived from a contract: renewal date, SLA, audit right, insurance certificate.
Risk Signal — External or internal data point that modifies supplier or contract risk posture.

Figure 10: Core Procurement Ontology

8.1 Bounded Context Map

Context	Aggregate Root(s)	Key Events	Store
Sourcing	SourcingEvent, Bid	RFxPublished · BidReceived · AwardDecided	PostgreSQL
Supplier	Supplier, Qualification	SupplierRegistered · QualificationApproved · RiskFlagRaised	PostgreSQL + Neo4j

Contract	Contract, Obligation	ContractDrafted · ContractSigned · ObligationDue	PostgreSQL + Vector
Purchasing	PurchaseRequisition, PO	PRCreated · POIssued · POAmended	PostgreSQL
Receiving	GoodsReceipt	GRNCreated · DiscrepancyFlagged	PostgreSQL
Invoicing	Invoice, MatchResult	InvoiceReceived · MatchFailed · ExceptionRaised	PostgreSQL
Payment	PaymentSchedule	PaymentApproved · PaymentScheduled	PostgreSQL
Risk	RiskProfile	RiskScoreUpdated · PolicyViolationDetected	PostgreSQL + Graph
Knowledge	DocumentChunk, Embedding	ChunkIndexed · EmbeddingRefreshed	PGVector + Blob
Marketplace	Plugin, PluginRelease	PluginPublished · PluginInstalled · PluginRevoked	PostgreSQL

Figure 11: Bounded Context Map

9. Data Architecture & Storage Responsibilities

Data Store Responsibility Matrix
PostgreSQL 15+ — System of record for all transactional domain entities. ACID, row-level security, schema-per-tenant.
PGVector — Co-located semantic vector store. Enables cosine-similarity search within ACID guarantees. Owns contract and document embeddings.
Neo4j — Relationship-centric knowledge graph. Supplier networks, contract obligation webs, risk signal propagation. Used for traversal queries impractical in SQL.
S3-Compatible Blob — Binary document storage. Contract PDFs, invoice images, qualification certificates. Immutable object versioning required.
ClickHouse — Immutable append-only audit store and analytics engine. All domain events, agent decisions, policy evaluations. INSERT ONLY — no DELETE API.
Redis — Session cache, rate limit counters, feature flag cache, pub/sub. TTL-governed throughout. Not a

system of record.
Kafka — Event streaming backbone. Durable, ordered, partitioned. Minimum 7-day retention. Cold archive to S3 after 7 days.

Figure 12: Data Store Responsibility Matrix

9.1 Data Lineage Event Schema

Data Lineage Event Schema
lineageId — UUID — unique identifier for this lineage record
sourceRefs — Array of {entityType, entityId, version, storeType} — input data sources
transformationRef — {serviceId, version, functionName} — processing logic reference (version-pinned)
outputRef — {entityType, entityId, version} — produced entity or document
actor — {type: HUMAN | AGENT, id, name, confidence?} — initiating actor
timestamp — ISO-8601 UTC nanosecond precision
traceId — OpenTelemetry trace ID for correlation with distributed traces

Figure 13: Data Lineage Event Schema

9.2 Data Classification Matrix

Every data type in OpenS2P is assigned a classification level that governs encryption, access control, retention, and AI handling rules. This matrix is the authoritative reference for all data handling decisions.

Classification Levels
PUBLIC — Freely shareable externally. No restriction. Examples: Published marketplace plugin descriptions, public API docs.
INTERNAL — For authorised users within a tenant. Not for external publication. Examples: Platform config, non-sensitive metadata.
CONFIDENTIAL — Business-sensitive. Restricted to authorised roles. Examples: Supplier pricing, PO values, invoice amounts, financial scores.
RESTRICTED — Highly sensitive. Regulatory requirements apply. Requires explicit access justification. Examples: Contracts, audit trails, personal data, payment data, secrets.

Data Type	Class	Encryption at Rest	Retention	PII in AI Prompts	Access Control
Supplier Master Record	CONFIDENTIAL	AES-256 (PG TDE)	7 years post-deactivation	ACL redaction required	Supplier Team + authorised buyers
Contract Document (PDF)	RESTRICTED	AES-256 (S3 SSE-KMS)	10 years + contract term	ACL redaction required	Contract Team + named signatories
Purchase Order	CONFIDENTIAL	AES-256 (PG TDE)	7 years	MUST NOT include PII fields	Procurement Team + supplier (sent copy)
Invoice	CONFIDENTIAL	AES-256 (PG TDE)	7 years	MUST NOT include PII fields	AP Team + supplier contact
Payment Schedule	RESTRICTED	AES-256 (PG TDE)	7 years	MUST NOT enter AI pipeline	AP Team + Finance Approver only
Goods Receipt (GRN)	INTERNAL	AES-256 (PG TDE)	7 years	Safe — no PII	Procurement Team + warehouse
LLM Prompt (assembled)	CONFIDENTIAL	AES-256 (in-transit)	90 days (prompt log)	PII redacted BEFORE assembly	AI Platform Team only
LLM Response (raw)	CONFIDENTIAL	AES-256 (in-transit)	90 days	Re-hydrated post-receipt	AI Platform Team only
Vector Embeddings	INTERNAL	AES-256 (PGVector)	Life of source document	Source doc PII already redacted	Knowledge Team (write) + agents (read)
Audit Trail Record	RESTRICTED	AES-256 (ClickHouse)	7 years — immutable	MUST NOT include raw PII	Compliance Team + auditors (read-only)
Session Token (JWT)	RESTRICTED	Signed RS256	TTL 15 min (auto-expire)	MUST NOT enter any store	Keycloak + individual user
Vault Secret	RESTRICTED	AES-256 (Vault seal)	TTL per lease (15 min–24h)	MUST NOT appear in logs or prompts	Vault (auto-revoke) + requesting service
Personal Data (GDPR Article 4)	RESTRICTED	AES-256 + tokenisation	Per consent; erasable	MUST be redacted — never enter LLM raw	Data subject + Privacy Officer

Supplier Financial Score	CONFIDENTIAL	AES-256 (PG TDE)	7 years	ACL redaction required	Risk Team + authorised leads

Figure R: Data Classification Matrix

9.3 Data Handling Rules

- RESTRICTED data MUST NOT appear in log output at any severity level. Logging frameworks MUST apply field-level masking before writing.
- RESTRICTED and CONFIDENTIAL data MUST NOT enter LLM prompts without passing through the PII Redactor. This is a platform constraint (C-12).
- Encryption keys for RESTRICTED data MUST be managed by Vault. Services MUST NOT hold keys in memory beyond the duration of a single operation.
- GDPR right-to-erasure MUST be implementable by deleting the tenant schema and associated S3 objects. No RESTRICTED personal data may be duplicated outside the authorised store without an explicit compliance decision.
- Audit trail records in ClickHouse MUST NOT contain raw personal data. Person references MUST use anonymised identifiers resolvable via a separate access-controlled lookup.

PART IV

AI ARCHITECTURE

10. Consolidated AI Architecture

OpenS2P is AI-native. AI capabilities are not bolt-on features — they are first-class architectural concerns expressed throughout every domain. This part consolidates the AI architecture into a single authoritative reference covering the agent model, prompt architecture standard, memory architecture, and AI evaluation framework.

⚠️	**AI Governance Principle** No AI agent in OpenS2P has unconditional authority to execute actions with business or financial consequence. Every agent chain has a configured confidence threshold. Below threshold, a mandatory human decision gate is invoked before execution proceeds.

10.1 Consolidated AI Reference Architecture

OpenS2P AI Reference Architecture — End-to-End

Trigger Layer
Event Trigger (Kafka) | API Trigger (REST/gRPC) | Scheduled Trigger (cron) | Human-Initiated (UI action)

Planning Layer
Task Decomposer | Tool Selector | Sub-Agent Planner | Confidence Threshold Initialiser

Retrieval Layer
Vector Search (PGVector — cosine similarity) | Graph Traversal (Neo4j — relationship paths) | Document Retrieval (S3 — blob fetch) | Live Data (Connector — real-time external)

Reasoning Layer
LLM Inference (vLLM self-hosted or external API) | Chain-of-Thought Synthesis | Structured Output Generation | Evidence Citation Mapper

Action Layer
Tool Calls → Domain Service APIs | Plugin Capability Invocation | Connector Egress | Workflow State Update

Validation Layer
Output Schema Validator | Business Rule Checker | Invariant Enforcement | Contradiction Detector

Confidence & Gate Layer
Aggregate Confidence Score (0.0–1.0) | Policy Compliance Check | Risk Signal Integration | Auto-Approve OR Human Escalation

Output & Audit Layer
Persist to System of Record | Emit Domain Events → Kafka | Write Immutable Audit Record → ClickHouse | Notify Downstream Subscribers

Figure 14: Consolidated AI Reference Architecture

10.2 Multi-Agent Interaction Model

Step	Actor	Action / Message	Target	Protocol
1	Contract Service	Emit ContractDrafted event with payload	Kafka	Kafka
2	Contract Analysis Agent	Consume event — begin analysis workflow via LangGraph	Workflow Orchestrator	gRPC
3	Retrieval Layer	Vector search for similar clauses in PGVector	PGVector	SQL
4	Retrieval Layer	Graph query for counterparty relationship history	Neo4j	Bolt
5	Reasoning Layer	LLM inference — clause extraction + risk flag generation	vLLM / LLM API	HTTPS
6	Contract Analysis Agent	Emit AnalysisComplete event with structured results	Kafka	Kafka
7	Risk Assessment Agent	Consume AnalysisComplete — initialise risk workflow	Workflow Orchestrator	gRPC
8	Risk Assessment Agent	Fetch supplier financial health from credit connector	Connector Runtime	gRPC
9	Risk Assessment Agent	Aggregate risk score — persist to Risk Profile	Risk Service	REST

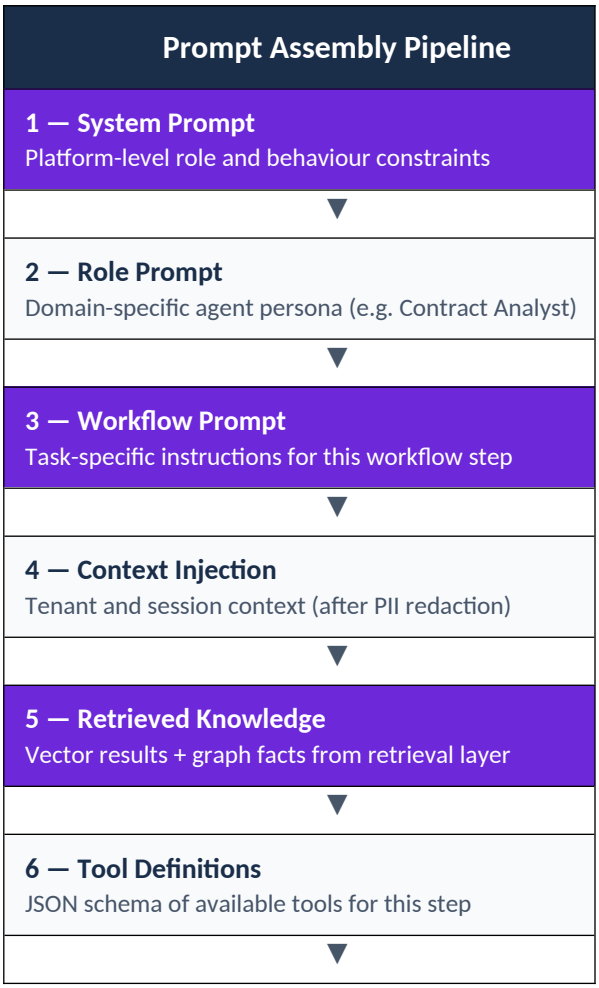
10	Confidence Gate	Score ≥ 0.92 → auto-approve; else escalate to human	Approval Gate	gRPC
11	Audit Layer	Write complete decision trace to ClickHouse	ClickHouse	TCP

Figure: Multi-Agent Handoff — Contract Analysis to Risk Assessment

Figure 15: Multi-Agent Handoff Sequence

11. Prompt Architecture Standard

OpenS2P defines a standard prompt architecture that applies to every agent and LLM invocation in the platform. This standard ensures that prompts are auditable, testable, PII-safe, and produce structured outputs that can be validated programmatically.



Prompt Architecture Rules

- Every prompt MUST include a system prompt that defines the platform safety constraints — these cannot be overridden by workflow prompts.
- Role prompts MUST be registered in the Prompt Registry and version-controlled. No ad-hoc role prompts are permitted in production.
- Every prompt that includes tenant data MUST pass through the PII Redactor before assembly. Placeholder tokens are re-hydrated in the response.
- Every prompt MUST include an Output Schema. Responses that do not conform to the schema are rejected and trigger a validation retry (maximum 2 retries before human escalation).
- Tool definitions MUST be the minimum set required for the workflow step. Providing excess tool definitions increases the attack surface for prompt injection.
- Prompts are immutable after version. Changes require a new version number

7 — Output Schema
Mandatory structured output format (JSON Schema)

and a regression evaluation run before production deployment.

Prompt Re
promptId — Reverse-domain string: com.opens2p.contract.clause-extracto
version — SemVer: MAJOR.MINOR.PATCH
domain — Bounded context this prompt belongs to
systemPromptRef — Reference to versioned system prompt
rolePromptText — Full role prompt text (version-controlled)
outputSchema — JSON Schema reference for structured output
evaluationRef — Reference to evaluation test suite for this prompt

Figure 16: Prompt Assembly Pipeline and Architecture Rules

12. Memory Architecture

Procurement AI requires persistent memory across sessions, workflows, and the organisation's history. OpenS2P defines three explicit memory layers with clear ownership, storage technology, TTL policy, and query semantics.

Memory Layer	Scope	What Is Stored	Store	TTL / Retention	Access Pattern
Session Memory	Single user session	Current workflow state, user intent context, in-progress decisions, partial results	Redis	Session TTL (24h max)	Read/write by active agent; cleared on session end
Workflow Memory	Single workflow execution	Complete agent execution graph state, tool call history, intermediate outputs, confidence scores, human decisions	PostgreSQL + LangGraph checkpoint	Configurable (default 90 days)	Read/write by workflow orchestrator; read-only after completion

Organisational Memory	Tenant-wide persistent	Historical contracts, supplier profiles, spend patterns, approved clause libraries, policy decisions, risk profiles	PGVector + Neo4j + PostgreSQL	Long-term (7 years per retention policy)	Read by retrieval layer during every agent invocation; write via domain services

Figure 17: Three-Layer Memory Architecture

12.1 Organisational Memory — Embedding Architecture

Organisational Memory Embedding Pipeline

Source Documents
Contracts (PDF/DOCX) | Supplier Qualifications | Historical POs/Invoices | Policy Documents | Regulatory Texts

Preprocessing
Text Extraction (OCR for scanned docs) | PII Detection and Redaction | Chunking Strategy (semantic paragraph boundaries) | Metadata Tagging (domain, entity IDs, dates)

Embedding Generation
Embedding Model (OpenAI text-embedding-3-large or local equivalent) | Batch Processing (async, rate-limited) | Version-tagged embeddings (model version pinned) | Tenant-namespaced storage

Storage & Refresh
PGVector (primary cosine search) | Neo4j (entity relationship context) | Refresh triggers: document update events | Stale embedding detection: version mismatch alert

Figure 18: Organisational Memory Embedding Pipeline

13. AI Evaluation Framework

AI quality is a measurable platform-level concern. Every agent in OpenS2P must be evaluated against the standard evaluation matrix before production deployment and on an ongoing basis in production. Evaluation results are stored in ClickHouse and monitored via Grafana dashboards.

	**Measurable AI Quality** The AI Evaluation Framework makes quality measurable. An AI agent is not 'done' when it

	produces outputs — it is done when it passes evaluation and its production metrics stay within SLO bounds. Evaluation is a continuous process, not a one-time gate.

13.1 Standard Evaluation Metrics Matrix

Metric	Definition	Target	Measurement Method	Frequency
Hallucination Rate	% of agent outputs containing factually incorrect statements unsupported by retrieved context	< 1%	LLM-as-judge evaluation on sampled outputs + human spot-check	Daily automated + weekly human review
Citation Coverage	% of AI recommendations that include traceable evidence references to source documents	100%	Automated check: every output parsed for evidence citations	Per-invocation, logged to ClickHouse
Tool Success Rate	% of tool call invocations that complete without error	≥ 99%	Automated: ClickHouse query on tool call results	Real-time, alerted on degradation
Workflow Completion Rate	% of agent workflows that reach a terminal success or human-decision state without dead-letter failure	≥ 95%	Automated: workflow state machine completion tracking	Real-time dashboard
Precision	% of AI-recommended actions that are correct (confirmed by human or downstream outcome)	≥ 95%	Retrospective: human decision override rate as proxy	Monthly cohort analysis
Recall	% of cases where a risk/issue actually exists that the AI correctly flagged	≥ 95%	Retrospective: audit of AI-missed flags vs reported issues	Monthly cohort analysis
Confidence Calibration	Correlation between predicted	Calibration error <	Expected Calibration Error	Per model version

	confidence score and actual correctness rate	5%	(ECE) on evaluation dataset	deployment
Latency (AI)	p95 end-to-end agent execution time (trigger to output)	< 30 seconds	OTEL trace spans on workflow execution	Real-time, dashboarded

Figure 19: AI Evaluation Metrics Matrix

13.2 Evaluation Pipeline Architecture

Step	Actor	Action / Message	Target	Protocol
1	Developer	Commit new/updated prompt version to Prompt Registry	Git / CI	Git
2	CI Pipeline	Trigger evaluation suite — offline golden dataset evaluation	Eval Runner	CI
3	Eval Runner	Run LLM-as-judge against 200+ domain-specific test cases	Eval LLM	HTTPS
4	Eval Runner	Compute hallucination rate, citation coverage, precision	Eval Report	Internal
5	CI Pipeline	Gate: all metrics within SLO? Allow merge. Else block.	GitHub PR	GitHub
6	Production Monitoring	Continuous evaluation: sample 5% of live invocations	Eval Sampler	Kafka
7	Eval Sampler	Write evaluation results to ClickHouse	ClickHouse	TCP
8	Grafana	Alert: metric degradation triggers PagerDuty	On-Call	Webhook

		alert		
9	On-Call	Review degradation — rollback prompt version if needed	Prompt Registry	REST

Figure: AI Evaluation Pipeline — Pre-Deployment and Production

Figure 20: AI Evaluation Pipeline — Pre-Deployment and Production Continuous

13.3 Agent Red Team Program

In addition to automated evaluation, OpenS2P operates a quarterly Agent Red Team program. Red team exercises probe agents for prompt injection vulnerabilities, goal misgeneralisation, unintended tool use, and exfiltration attempts. Red team findings feed directly into security hardening of the PII redactor, output validator, and prompt guard components.

Red Team Test Category	What Is Probed	Control Tested	Remediation Target
Prompt Injection	Malicious user content attempting to override system prompt	System prompt immutability, output schema enforcement	PII Redactor + Prompt Guard
Tool Abuse	Agent instructed to call tools outside its declared capability scope	Capability Guard (OPA policy)	Plugin Capability Guard
Data Exfiltration	Agent instructed to include sensitive data in observable output fields	PII Redactor, output schema, egress filter	PII Redactor hardening
Goal Misgeneralisation	Edge case inputs designed to make agent pursue an incorrect goal	Output schema validation, business rule checker	Validator + Eval dataset expansion
Confidence Gaming	Crafted inputs that produce artificially high confidence on incorrect outputs	Confidence calibration check, human gate threshold	Calibration monitoring + threshold tuning

Figure 21: Agent Red Team Program Test Categories

13.4 AI Cost Governance

AI inference is one of the highest and most variable cost drivers in an AI-native platform. OpenS2P defines a formal AI Cost Governance model that is enforced at the platform level — not left to individual services or tenants to manage. Enterprise AI architects SHOULD review this section before production deployment.

Control	Mechanism	Scope	Configuration Level
Prompt Cache	Identical prompt prefixes	Per agent type	Platform default; per-

	served from cache (vLLM prefix caching or provider-level). Avoids re-tokenising static system + role prompts on every request.		tenant override
Token Budget	Maximum input + output token limit per agent invocation type. Hard limit enforced by prompt assembly layer before LLM call. Prevents runaway context window costs.	Per workflow step	ADR + operator config
Embedding Reuse	Document embeddings are cached by content hash. Re-embedding is only triggered on document change (version bump event). Prevents duplicate embedding charges.	Per document	Automatic — event-driven
Model Selection Policy	Agent workflows declare a minimum model capability tier (FAST / STANDARD / ADVANCED). The model router selects the cheapest model that meets the tier. Default: STANDARD.	Per workflow	Workflow manifest
Model Routing	Router evaluates: (1) self-hosted vLLM availability and queue depth, (2) latency SLO, (3) cost per token. Routes to self-hosted first; falls back to API on capacity exhaustion.	Platform-wide	Operator config
GPU Scheduling	GPU node pool uses spot/preemptible instances where SLO permits. Agent workflows declare latency class: INTERACTIVE (guaranteed GPU) or BATCH (spot eligible).	Per workflow	Workflow manifest + K8s node affinity

Daily Token Budget Alerts	Per-tenant daily token consumption tracked in ClickHouse. Alert fires at 80% of configured daily budget. Hard cutoff at 100% (configurable per tenant).	Per tenant	Tenant admin config
AI Spend Dashboard	Grafana dashboard: token cost by tenant, by agent, by model, by day. Cost allocation tags on every LLM API call for FinOps reporting.	Platform-wide	Pre-built; no config required

Figure J: AI Cost Governance Controls

PART V

RUNTIME AND INTERACTION ARCHITECTURE

14. Workflow Orchestration Model

OpenS2P uses LangGraph as its workflow orchestration engine (ADR-003). LangGraph models procurement workflows as directed graphs where nodes represent processing steps and edges represent transitions conditioned on state evaluation. This enables conditional branching, parallel fan-out/fan-in, human gate suspension and resumption, and compensating transaction paths.

Step	Actor	Action / Message	Target	Protocol
1	Client	POST /workflow/execute with payload and workflow definition ID	API Gateway	HTTPS
2	API Gateway	Authenticate JWT, evaluate OPA policy for workflow:execute scope	OPA	gRPC
3	Workflow Orchestrator	Load graph definition, initialise LangGraph state	Graph Registry	Redis
4	Planning Node	Decompose task, select tools, set confidence threshold	Tool Registry	gRPC
5	Retrieval Node	Vector search + graph traversal for context	PGVector / Neo4j	SQL/Bolt
6	Agent Executor	Invoke LLM with assembled prompt (post-PII-redaction)	LLM Service	HTTPS
7	Validation Node	Parse output against JSON Schema, check business rules	Validator	Internal
8	Confidence Node	Compute aggregate confidence score	Confidence Scorer	Internal
9	Gate Node	Score ≥ threshold → proceed; else suspend and notify approver	Approval Gate	gRPC

10	Output Node	Write results, emit events, write ClickHouse audit record	Domain Service / Kafka / ClickHouse	Multi

Figure: Agent Request Full Execution Flow

Figure 22: Agent Request Execution — Full Sequence

15. Standardised Agent Lifecycle

OpenS2P Standardised Agent Lifecycle — Nine Stages

Stage 1 — Input Trigger
Event / API / Scheduled / Human-initiated

Stage 2 — Planner
Task decomposition · Tool selection · Escalation path definition

Stage 3 — Retriever
Vector search (PGVector) · Graph traversal (Neo4j) · Document fetch (S3) · Live data (Connectors)

Stage 4 — Reasoner
LLM inference · Chain-of-thought · Evidence synthesis · Structured output

Stage 5 — Tool Calls
Domain service API writes · Plugin invocation · Connector egress · State update

Stage 6 — Validator
Schema validation · Business rule check · Invariant enforcement · Contradiction detection

Stage 7 — Confidence Scorer
Evidence quality aggregate · Model uncertainty · Policy compliance · Risk signal integration

Stage 8 — Approval Gate
Confidence ≥ threshold → auto-approve · Below → human escalation · Mandatory audit record

Stage 9 — Output & Audit
System of record write · Kafka events · ClickHouse immutable audit · Downstream notification

Figure 23: Standardised Agent Lifecycle — Nine-Stage Model

15.1 Human-in-the-Loop Gate Model

HITL Gate Configuration Model
Auto-Approve Threshold — Configurable per workflow type. Default: 0.92. Below this score → mandatory human review.
Escalation Target — Role-based routing. Determined by spend band, category policy, and action type.
Timeout Policy — No response within SLA window (default 48h) → escalate to secondary approver or auto-reject.
Override Audit — Human override of AI recommendation requires mandatory reasoning field. Stored in ClickHouse.
Override Analytics — Override patterns feed back into evaluation framework to identify systematic AI calibration gaps.

Figure 24: Human-in-the-Loop Gate Configuration Model

PART VI

EXTENSIBILITY AND ECOSYSTEM

16. Plugin Runtime Architecture

Plugins extend the functional capabilities of the OpenS2P platform. Unlike connectors — which are integration adapters — plugins are arbitrary code bundles that can add new agent tools, UI widgets, workflow steps, or approval logic. Plugins require strict isolation because they run arbitrary, potentially untrusted code (ADR-007).

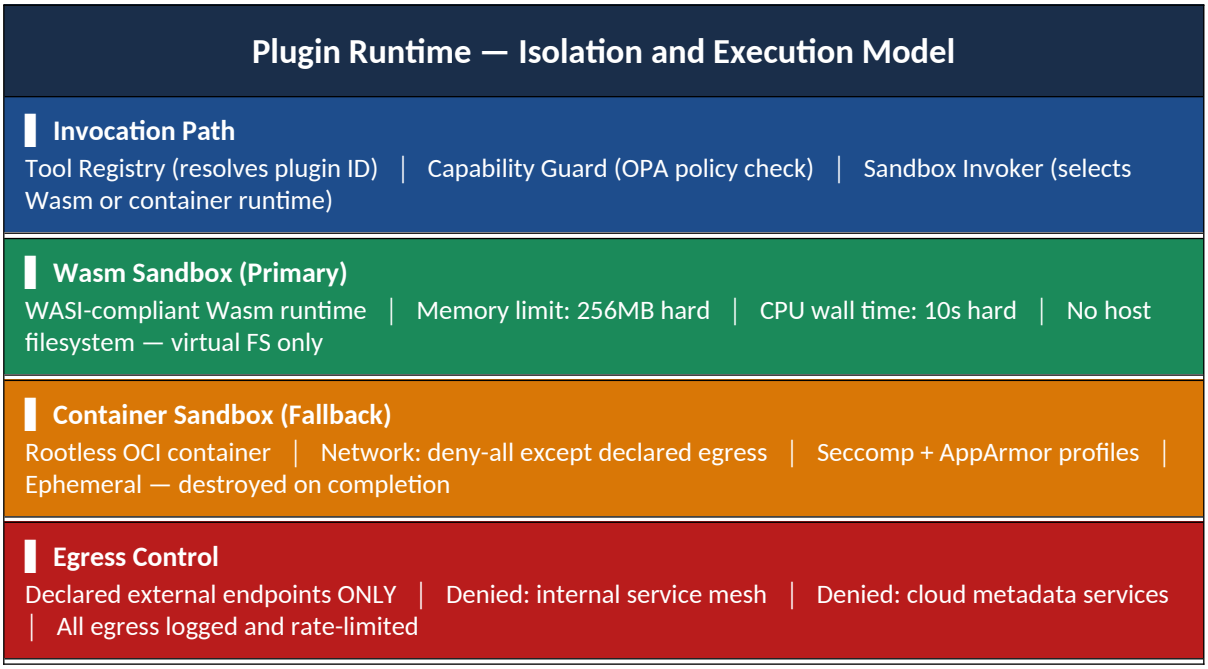


Figure 25: Plugin Runtime — Isolation Architecture

17. Marketplace Architecture & Trust Model

Step	Actor	Action / Message	Target	Protocol
1	Publisher	Build plugin, sign with private key, submit to Marketplace	Marketplace API	HTTPS
2	Trust Service	Validate manifest, verify signature, record checksum in provenance chain	Provenance Store	gRPC
3	Scanner	Automated SAST + dependency vulnerability audit	Scanner Service	Internal
4	Moderator	Human review — approve or reject	Marketplace	HTTPS

		plugin release	Console	
5	Marketplace	Publish to CDN, update version index	CDN / Registry	HTTPS
6	Tenant Admin	Install plugin — platform verifies allowlist + compatibility matrix	Marketplace API	HTTPS
7	Plugin Sandbox	Download, verify checksum, deploy to sandbox, health check	Tool Registry	gRPC
Figure: Plugin Publication and Installation Flow				

Figure 26: Plugin Publication and Installation Sequence

PART VII

SECURITY AND OPERATIONS

18. Security Reference Architecture

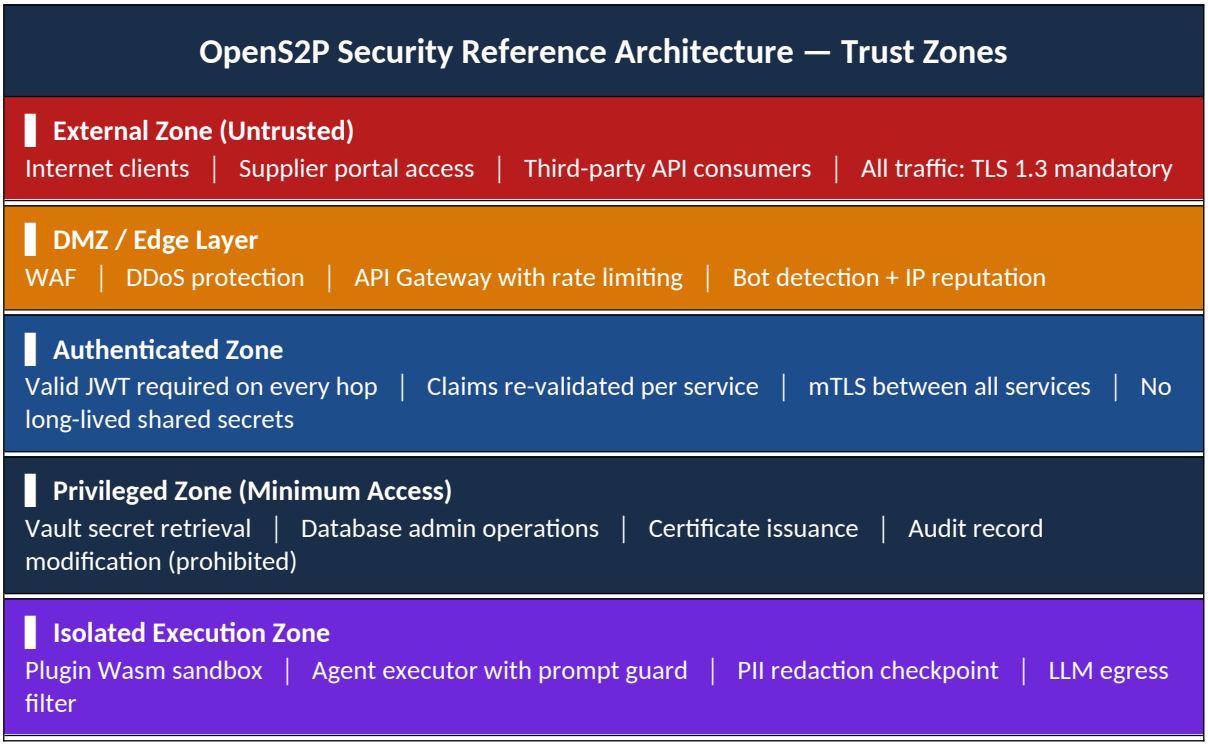


Figure 27: Security Reference Architecture — Trust Zone Model

18.1 Identity Trust Chain

Step	Actor	Action / Message	Target	Protocol
1	User Browser	Navigate — redirect to enterprise IdP	Keycloak	OIDC
2	Enterprise IdP	Authenticate user (MFA), return SAML assertion	Keycloak	SAML 2.0
3	Keycloak	Map groups to roles, issue short-lived JWT (15 min TTL)	User Browser	OIDC
4	API Gateway	Validate JWT signature, extract claims, attach to context	Domain Service	HTTPS
5	Domain Service	Delegate authorisation	OPA	gRPC

		evaluation to OPA sidecar		
6	Domain Service	Request dynamic DB credential from Vault	Vault	HTTPS
7	Vault	Issue time-limited PostgreSQL credential (15 min TTL)	Domain Service	HTTPS

Figure: Enterprise SSO to Service Credential Flow

Figure 28: Identity Trust Chain — SSO to Service Credential Flow

19. Deployment Topology & Environment Profiles

19.1 SaaS Multi-Tenant Production Topology

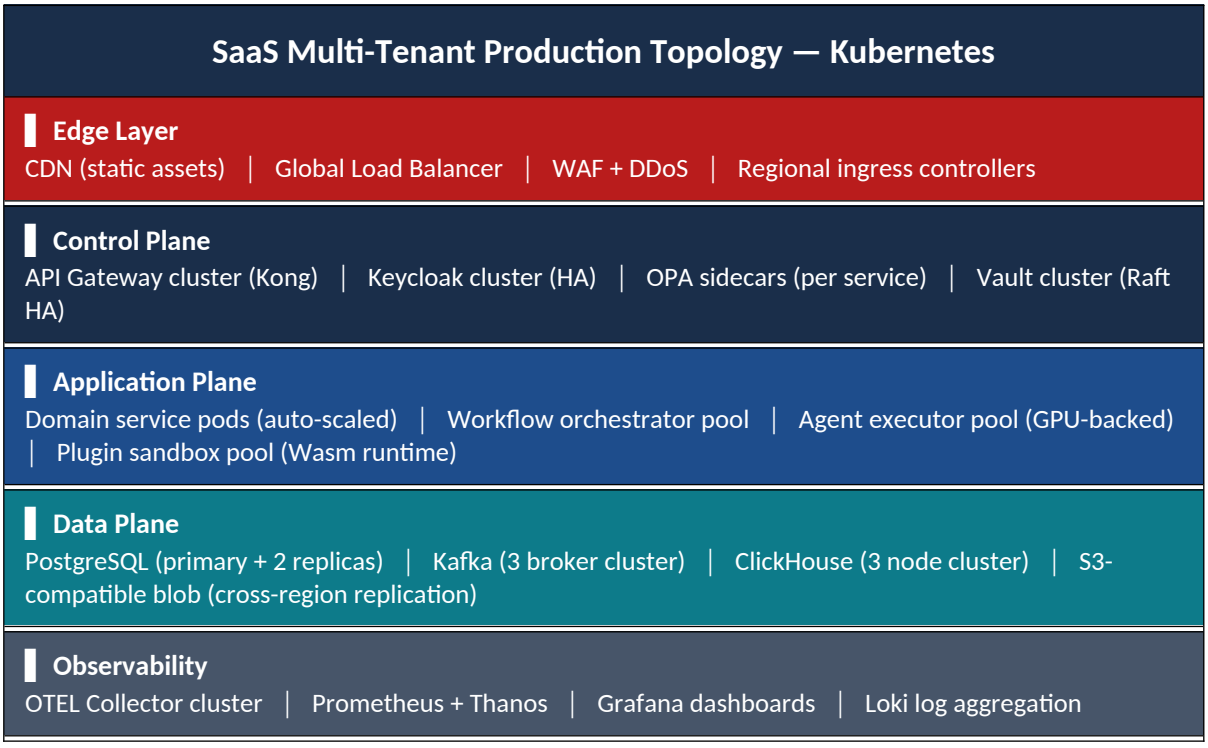


Figure 29: SaaS Multi-Tenant Production Topology

19.2 Enterprise Air-Gapped Topology

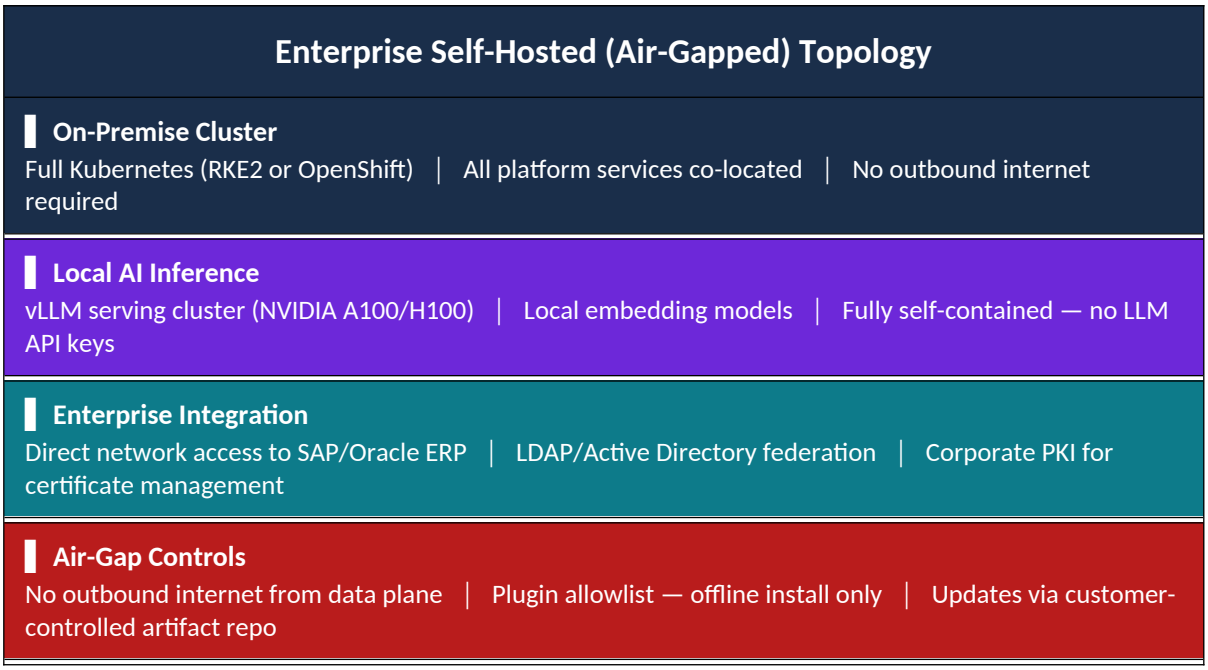


Figure 30: Enterprise Air-Gapped Deployment Topology

19.3 Reference Deployment Profiles

OpenS2P defines three reference deployment profiles. These profiles are not hard boundaries — they are calibrated starting points for capacity planning. Organisations should use these as baselines and adjust based on their specific tenant count, transaction volumes, and AI workload intensity.

STARTER PROFILE	
Target	SMB · Single tenant · Pilot deployments · Non-production
Compute	4 vCPU · 16 GB RAM · 3 Kubernetes nodes (control + 2 worker)
Data	Single PostgreSQL instance · Single Kafka broker · Redis single node · S3-compatible blob (50 GB)
AI	External LLM API (OpenAI / Anthropic) OR Ollama on worker node · No GPU required
Capacity	≤ 5 tenants · ≤ 10,000 suppliers · ≤ 500K invoices/year · ≤ 50 AI requests/min · ≤ 100 concurrent users

Figure F: Starter Deployment Profile

ENTERPRISE PROFILE	
Target	Enterprise self-hosted · Regulated industries · Air-gapped options · Full HA
Compute	32–64 vCPU · 128–256 GB RAM · HA Kubernetes cluster (6+ nodes) · Dedicated node pool per layer
Data	PostgreSQL HA (primary + 2 replicas) · Kafka 3-broker cluster · Neo4j single/cluster · ClickHouse 3-node · Redis HA · S3 (multi-TB)
AI	vLLM on GPU pool (2–4× NVIDIA A100 or H100) · Local embedding model · Air-gap capable
Capacity	≤ 100 tenants · ≤ 1M suppliers · ≤ 50M invoices/year · ≤ 500 AI requests/min · ≤ 5,000 concurrent users

Figure G: Enterprise Deployment Profile

LARGE ENTERPRISE / SAAS PROFILE	
Target	SaaS platform · Large enterprise · Multi-region · 100K+ users · Global supplier networks
Compute	Multi-region Kubernetes (3+ regions) · 64–256+ vCPU per region · Auto-scaling node pools · CDN edge layer
Data	PostgreSQL global cluster (read replicas per region) · Kafka multi-region MirrorMaker 2 · ClickHouse distributed cluster · S3 cross-region replication · 10TB+ document storage
AI	GPU pool per region (8–16× H100) · Model router (self-hosted + external API fallback) · Distributed embedding pipeline · Prompt cache layer
Capacity	≤ 10,000 tenants · ≤ 10M suppliers · ≤ 100M invoices/year · ≤ 1,000 AI requests/min · ≤ 100,000 concurrent users · 500M+ audit records

Figure H: Large Enterprise / SaaS Deployment Profile

19.4 Capacity Planning Reference

The following capacity planning matrix maps expected data volumes to storage tier requirements. These figures are derived from procurement industry benchmarks and platform stress testing. Organisations SHOULD validate against their actual transaction volume before production sizing.

Data Category	Starter	Enterprise	Large Enterprise	Storage Tier
Active tenants	≤ 5	≤ 100	≤ 10,000	Keycloak + PG schema
Supplier records	≤ 10,000	≤ 1,000,000	≤ 10,000,000	PostgreSQL + Neo4j
Invoices per year	≤ 500,000	≤ 50,000,000	≤ 100,000,000	PostgreSQL + ClickHouse
Document storage	≤ 50 GB	≤ 10 TB	≤ 100 TB	S3-compatible blob
Audit records	≤ 5M	≤ 500M	≤ 5B	ClickHouse
Vector embeddings	≤ 1M chunks	≤ 100M chunks	≤ 1B chunks	PGVector
AI requests / minute	≤ 50	≤ 500	≤ 1,000+	vLLM / LLM API
Concurrent users	≤ 100	≤ 5,000	≤ 100,000	API Gateway + K8s autoscale
Kafka throughput	≤ 1 MB/s	≤ 50 MB/s	≤ 500 MB/s	Kafka cluster sizing
Graph nodes (Neo4j)	≤ 100K	≤ 10M	≤ 500M	Neo4j cluster

Figure I: Capacity Planning Reference Matrix

19.5 Disaster Recovery Strategy

The RTO / RPO targets defined in Chapter 4 require a concrete recovery playbook for each critical platform component. The following table defines the DR strategy per component. Recovery procedures SHOULD be rehearsed on a quarterly basis as part of the chaos engineering program.

Component	Failure Scenario	Recovery Strategy	RTO Target	RPO Target	Test Frequency
PostgreSQL	Primary instance failure	Automatic failover to replica via Patroni / CloudSQL HA. DNS updated by	< 60s	< 5 min	Monthly

		patroni leader election. Application reconnects via PgBouncer connection pooler.			
Kafka	Broker failure	Kafka replication factor 3. Surviving brokers continue serving. Failed broker replaced; partition reassignment automatic.	< 30s (degraded)	Zero (replicated)	Monthly
ClickHouse	Node failure	ClickHouse distributed cluster with shard replication. Reads and inserts continue via surviving replicas. Failed node rejoins on restart.	< 5 min (degraded)	Zero (replicated)	Quarterly
Vault	Vault node failure	Vault Raft HA (3 nodes). Automatic leader re-election. Sealed state requires operator unseal or auto-unseal via cloud KMS.	< 30s	Zero (Raft log)	Monthly
Keycloak	Instance failure	Keycloak HA cluster with shared database (PostgreSQL). Load balancer routes to surviving instances. Session stickiness not	< 30s	Zero (JWT stateless)	Monthly

		required (JWT stateless).			
OPA	Policy sidecar crash	OPA runs as sidecar on each service pod. Kubernetes restarts crashed sidecar. Requests during restart window: fail-closed (deny).	< 10s (Kubernetes restart)	N/A (policy from bundle)	Per deploy
Region failure	Complete regional outage	DNS failover to secondary region. PostgreSQL global replica promoted. Kafka MirrorMaker 2 consumer groups re-pointed. RTO depends on data replication lag.	< 1 hour	< 30 min	Semi-annual
S3 / Blob store	Object store unavailability	Cross-region replication active. Read traffic redirected to replica region. Write traffic queued in Kafka until primary restored.	< 15 min	Zero (cross-region sync)	Quarterly

Figure K: Disaster Recovery Strategy by Component

18.3 Integration Pattern Catalog

This catalog defines the standard integration patterns for OpenS2P service developers. Every integration MUST use one of these named patterns. Ad-hoc approaches require a TSC-approved ADR before implementation.

Integration Patterns — Summary

Pattern 1 — Publish/Subscribe (MUST for cross-domain events)
Default for all cross-domain communication. Publisher emits to Kafka topic. Downstream consumers subscribe independently. AsyncAPI schema required.

Pattern 2 — Request/Reply (intra-domain or OHS only)
Synchronous query within a bounded context OR calling a designated Open Host Service. MUST NOT be used for cross-domain state-changing integration.

Pattern 3 — Event-Carried State Transfer (SHOULD where payload < 1MB)
Event payload includes enough data for downstream to act without callback. Reduces coupling. Events MUST NOT include blobs — reference S3 object key instead.

Pattern 4 — Outbox Pattern (MUST for state change + event)
Write to domain table + outbox table in same PostgreSQL transaction. Debezium CDC relay reads outbox and publishes to Kafka. Guarantees atomicity.

Pattern 5 — CQRS (SHOULD for complex read models)
Write side: domain service + PostgreSQL aggregate. Read side: materialised projection updated by consuming events. Used by Spend Analytics context.

Pattern 6 — Saga (SHOULD for distributed transactions)
Business process spanning multiple contexts. Each step publishes success or failure event. Choreography-based (preferred) or orchestration-based (LangGraph for complex flows).

Pattern 7 — Change Data Capture / CDC (MAY, outbox relay only)
Debezium reads PostgreSQL WAL. Publishes to internal Kafka topics. Domain services MUST NOT consume CDC topics directly — they consume domain event topics only.

Figure Q: Integration Pattern Catalog

Pattern	Direction	Coupling	When to Use	When NOT to Use	RFC 2119
Publish/ Subscribe	Async 1:N	Loose	All cross-domain business events	When publisher needs synchronous response	MUST (cross-domain)
Request/Reply	Sync 1:1	Tight	OHS calls, intra-domain queries	Cross-domain state-changing integration	MUST NOT (cross-domain state change)
Event-Carried State Transfer	Async enriched	Medium	Self-contained events < 1MB payload	Large payloads or rapidly changing embedded state	SHOULD

Outbox Pattern	Async guaranteed	Loose	All state change + event publishing	Read-only query handlers	MUST (state change + event)
CQRS	Async read projection	Loose	Complex reporting, analytics, search	Simple CRUD in single context	SHOULD
Saga	Async distributed TX	Loose	Multi-domain business processes	Single-domain operations	SHOULD
CDC	Async DB capture	Medium	Outbox relay, analytics sync only	Primary inter-service integration	MAY (outbox relay only)

Figure Q2: Integration Pattern Selection Matrix

PART VIII

TESTING ARCHITECTURE

20. Testing Architecture

Enterprise platforms require a comprehensive, layered testing architecture. OpenS2P defines eight testing disciplines that together provide confidence from individual functions through to full production resilience.

OpenS2P Testing Architecture — Eight Disciplines

Unit Testing
Domain logic — entities, aggregates, value objects, domain events | Pure function tests — no database, no network | Coverage target: 90% line coverage on domain layer | Tools: Jest (TypeScript), Pytest (Python)

Integration Testing
Infrastructure layer — repositories, event bus adapters, external clients | Real database (TestContainers), stubbed external network | Coverage target: all repository methods + event schema serialisation | Tools: Testcontainers, Supertest

Contract Testing
Consumer-driven contracts between all service pairs | Pact provider verification in CI — breaks deployment if contract violated | Event schema contracts: AsyncAPI-driven consumer tests | Tools: Pact.io, AsyncAPI test generators

Load & Performance Testing
API latency targets: p95 < 400ms under 1,000 concurrent users | Agent throughput: 500+ concurrent executions per node | Database query performance: p99 < 100ms for standard queries | Tools: k6, Locust, Gatling

Security Testing
SAST in CI (Semgrep, CodeQL) — blocks merge on critical findings | DAST in staging (OWASP ZAP) — weekly automated scans | Dependency audit in CI — CVE blocking gate: HIGH severity | Penetration test: annual external engagement

AI Evaluation Testing
Offline evaluation: golden dataset (200+ domain cases per agent) | LLM-as-judge: automated hallucination + citation coverage scoring | Prompt regression: new prompt version must pass baseline before merge | Confidence calibration: Expected Calibration Error < 5%

Red Team Testing
Quarterly adversarial agent testing: prompt injection, tool abuse, exfiltration | Findings catalogued and fed to Prompt Guard + PII Redactor hardening | Out-of-scope actions: model cannot be tasked with red team against production

Chaos Engineering
Monthly chaos experiments: random pod termination, network partition | Database failover drills: verify < 5-minute RPO | Agent timeout simulation: verify dead-letter handling and HITL escalation | Tools: Chaos Mesh, Litmus, AWS Fault Injection Simulator

Figure 31: OpenS2P Testing Architecture — Eight Disciplines

20.1 Test Pipeline Architecture

Step	Actor	Action / Message	Target	Protocol
1	Developer	Push code — PR opened	GitHub	Git
2	CI	Unit tests (Jest/Pytest) — gate: 0 failures, 90% coverage	Test Runner	CI
3	CI	SAST scan (Semgrep/CodeQL) — gate: no HIGH/CRITICAL findings	Scanner	CI
4	CI	Dependency audit — gate: no HIGH CVEs unpatched > 24h	Audit Tool	CI
5	CI	Contract test verification — gate: all Pact contracts pass	Pact Broker	HTTPS
6	CI	AI evaluation (if prompt changed) — gate: all metrics within SLO	Eval Runner	CI
7	CI	Integration tests — gate: 0 failures	Testcontainers	CI
8	Staging	DAST scan — gate: no NEW medium/high findings	OWASP ZAP	HTTPS
9	Staging	Load test — gate: p95 latency < 400ms at target concurrency	k6	HTTPS
10	Production	Canary deploy 5% traffic — monitoring window	Argo Rollouts	Kubernetes

		30 min		
11	Production	Auto-promote or rollback based on error rate SLO	Argo Rollouts	Kubernetes

Figure: CI/CD Testing Pipeline — Gate Structure

Figure 32: CI/CD Testing Pipeline Gate Structure

20.2 Testing Ownership Matrix

Test Type	Owner	Environment	Blocking Gate	Frequency
Unit tests	Feature developer	Local + CI	PR merge	Every commit
Integration tests	Feature developer	CI (Testcontainers)	PR merge	Every commit
Contract tests	Consuming team	CI (Pact Broker)	PR merge	Every commit
SAST	Platform Security	CI	PR merge	Every commit
Dependency audit	Platform Security	CI	PR merge	Every commit + daily scan
AI evaluation	AI Platform team	CI + production	PR merge (prompt changes)	Every prompt change + daily sample
Load tests	SRE team	Staging	Release promotion	Per release + weekly
DAST	Platform Security	Staging	Release promotion	Weekly
Penetration test	External (contracted)	Staging/Production	Annual release	Annual
Red team	AI Safety team	Staging	Quarterly review	Quarterly
Chaos engineering	SRE team	Production (off-peak)	Post-incident review	Monthly

Figure 33: Testing Ownership Matrix

20.3 CI/CD Reference Architecture

All OpenS2P platform services MUST follow this reference CI/CD pipeline. The pipeline enforces all testing gates from Chapter 20 and adds supply chain security controls (SBOM, image signing) required for enterprise deployments.

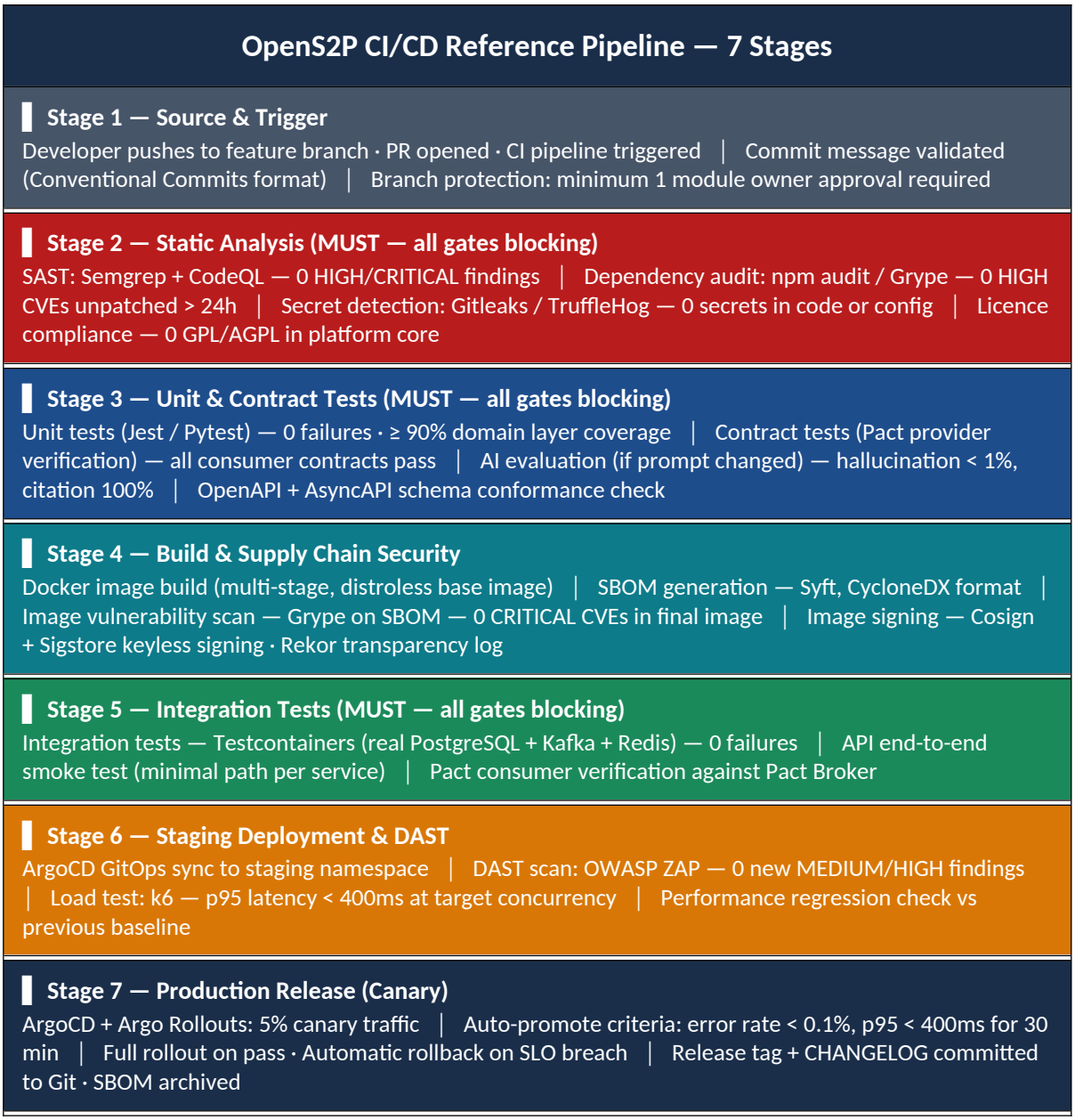


Figure S: CI/CD Reference Pipeline — 7-Stage Gate Architecture

20.4 Supply Chain Security Controls

Control	Tool	Stage	What It Prevents	Gate Type
Secret detection	Gitleaks / TruffleHog	Pre-commit + CI Stage 2	Credentials committed to source	MUST — blocks PR merge
SAST	Semgrep + CodeQL	CI Stage 2 (every commit)	Known vulnerability patterns in source	MUST — blocks on HIGH+

Dependency audit	npm audit / Grype on SBOM	CI Stage 2 + daily scan	Known CVEs in transitive dependencies	MUST — HIGH+ unpatched > 24h blocks
SBOM generation	Syft (CycloneDX)	CI Stage 4 — image build	No bill of materials for audit	MUST — no SBOM = no release
Image signing	Cosign + Sigstore	CI Stage 4 — image build	Tampered images deployed to production	MUST — unsigned images rejected by admission controller
Image scanning	Grype against SBOM	CI Stage 4 — image build	CRITICAL CVEs in final image	MUST — CRITICAL = 0 for production
Licence compliance	licence-checker / FOSSA	CI Stage 2	GPL/copyleft licence contamination	MUST — GPL in core = blocked
Admission controller	Kyverno / OPA Gatekeeper	Kubernetes runtime	Unsigned or policy-violating images in production	MUST — deployed as cluster policy

Figure T: Supply Chain Security Controls

PART IX

COMPLIANCE ARCHITECTURE

21. Compliance Architecture

OpenS2P is designed to support regulated enterprise procurement. Compliance is not a checklist bolted on post-deployment — it is an architectural concern built into the audit model, data retention policy, access control design, and AI governance model from the ground up.

Compliance Philosophy

OpenS2P provides the architectural foundations for compliance. Achieving a specific certification (SOC 2 Type II, ISO 27001, etc.) requires the deploying organisation to also establish the operational controls, policies, and evidence procedures specific to their environment and certification target.

21.1 Compliance Framework Coverage Matrix

Compliance Framework	Relevant To	Key Platform Controls	Coverage Level
SOX (Sarbanes-Oxley)	Public companies with procurement > financial reporting	Segregation of duties via OPA ABAC · Immutable audit trail (ClickHouse) · Approval workflow with mandatory audit record · Change management via ADR governance	Core controls — evidence generation automated
ISO 27001	All enterprise deployments	Zero-trust security model · Secrets management (Vault) · Vulnerability management (CI gate) · Access control (OPA + Keycloak) · Incident response workflow	Control framework alignment — gap assessment available
SOC 2 Type II	SaaS deployments	Availability SLOs monitored · Security controls documented · Confidentiality via tenant isolation · Processing integrity via audit trail · Privacy via PII redaction	Trust service criteria mapped — audit evidence automated
GDPR	EU data subjects	PII detection and redaction pre-LLM · Data residency controls (tenant partitioning) · Right-to-erasure: tenant schema deletion · Data lineage tracking · Consent management	Data subject rights architecturally supported

		integration point	
NIST AI RMF	AI-native platform	AI governance framework (Chapter 13) · Confidence scoring and human oversight · Red team program · Bias monitoring hooks · Model card registry	Framework alignment — implementation roadmap in v5.0
ISO/IEC 42001	AI management systems	AI evaluation framework · Prompt registry with versioning · AI incident response workflow · Governance model for AI changes · Traceability from output to evidence	Partial — full certification path in v5.x roadmap
HIPAA	Healthcare procurement packs (optional)	Encryption at rest and in transit · Access logging to ClickHouse · Minimum necessary access (OPA) · BAA framework at API boundary · Audit report generation	Plugin pack — optional module; not in core scope
PCI-DSS	Payment data handling	Tokenisation at payment rail boundary · No card data stored in OpenS2P · Audit log retention · Access control to payment schedules	Boundary control — PCI scope excluded from core

Figure 34: Compliance Framework Coverage Matrix

21.2 Audit Evidence Architecture

The audit evidence architecture is designed to satisfy the most demanding regulatory audit requirements without bespoke reporting work. The ClickHouse audit store captures a complete, tamper-evident record of every business event, AI decision, policy evaluation, and access control decision.

Audit Evidence Architecture — Data Sources and Evidence Packs

Evidence Sources

Domain Events (all business state changes) | Agent Decision Records (9-stage lifecycle captured) | Policy Evaluation Logs (every OPA allow/deny) | Access Control Audit (every authenticated request) | Human Approval Records (HITL decisions with reasoning)

ClickHouse Audit Store

Insert-only — no DELETE or UPDATE API | Tamper-evidence via append-only storage policy |

Partitioned by tenant, domain, and date | Compressed columnar storage — cost-effective at 7-year retention

Evidence Pack Generator

On-demand regulatory evidence pack generation | Filters by: date range, entity ID, user, domain, event type | Export formats: PDF, CSV, JSON, signed archive | Pre-built templates: SOX, ISO 27001, SOC 2, GDPR

Audit Access Controls

Audit API: read-only, requires explicit audit:read scope | Auditor role: separate from admin — cannot modify records | External auditor access: time-limited, scoped credentials | Evidence pack integrity: SHA-256 checksum + signing

Figure 35: Audit Evidence Architecture

21.3 AI Regulatory Alignment

Regulation / Standard	AI-Specific Requirement	OpenS2P Control
EU AI Act (High Risk Systems)	Transparency, logging, human oversight for high-risk AI	HITL gates · Confidence scoring · Audit trail of all AI decisions · Model documentation via prompt registry
NIST AI RMF — Govern	Policies and accountability for AI risk	AI governance model · Red team program · TSC oversight of AI architecture changes
NIST AI RMF — Map	Identify AI risks in context of use	Threat model extended to AI attacks (Chapter 18) · Red team test categories documented
NIST AI RMF — Measure	Ongoing monitoring of AI performance	AI evaluation framework · Production metric dashboards · Drift detection alerts
NIST AI RMF — Manage	Respond to AI incidents	AI incident response workflow · Prompt rollback mechanism · Emergency agent disable toggle
ISO/IEC 42001	AI management system	Prompt registry (versioned) · Evaluation framework · Governance model documents AI authority boundaries

Figure 36: AI Regulatory Alignment Matrix

PART X

GOVERNANCE AND EVOLUTION

22. Architecture Governance Model

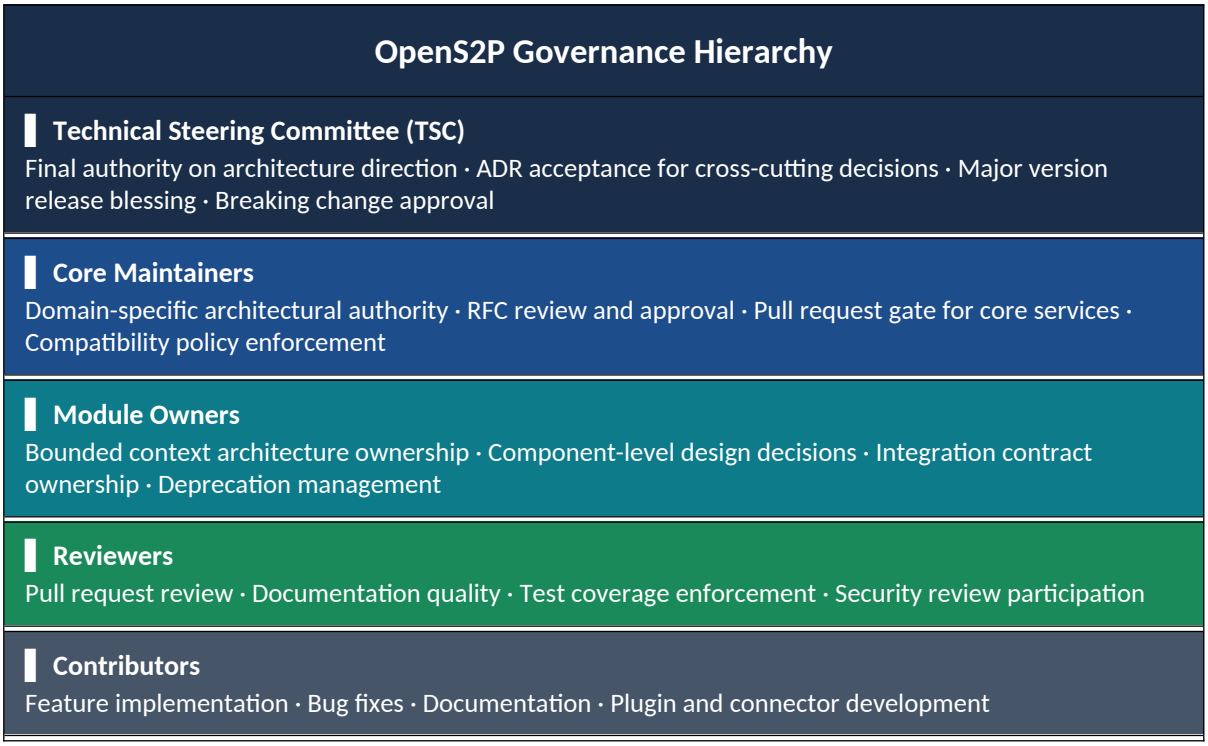


Figure 37: OpenS2P Governance Hierarchy

22.1 RFC and ADR Workflow

Step	Actor	Action / Message	Target	Protocol
1	Contributor	Open GitHub issue with architecture-proposal label	GitHub Issues	GitHub
2	Module Owner	Triage issue — assign RFC track or reject with reason	GitHub Issues	GitHub
3	Contributor	Draft RFC using template in /docs/rfcs/	GitHub PR	GitHub
4	Core Maintainers	Review RFC — 14-day minimum public comment period	GitHub PR	GitHub
5	TSC	Accept, request changes, or reject	GitHub PR	GitHub

		RFC		
6	Contributor	Convert accepted RFC to ADR format	GitHub PR	GitHub
7	TSC	Mark ADR status as Accepted after implementation verified	ADR Catalog	GitHub

Figure: Architecture Proposal — Issue to Accepted ADR

Figure 38: Architecture Proposal Lifecycle

23. ADR Catalog

The following 16 Architecture Decision Records form the decision history of the OpenS2P platform. ADRs are immutable once Accepted. Superseded ADRs are preserved for historical context.

ADR-001: Use C4 Model as Canonical Architecture Notation

ID	ADR-001
Status	Accepted
Review	Annual review — 2027-Q1

Context

Contributors needed a consistent, widely understood notation for communicating architecture at multiple levels of abstraction. Ad-hoc diagrams were inconsistent and made it difficult for new contributors to navigate the system.

Decision

C4 model (Context, Container, Component, Code) is adopted as the mandatory notation for all architecture documentation. Every major subsystem must be described at the appropriate C4 level before lower-level implementation guidance is provided.

Alternatives Considered

- UML alone — powerful but steep learning curve
- ArchiMate — enterprise-grade but heavyweight for OSS contributors
- Informal box-and-line — lowest barrier but no consistency

Consequences

- All architectural documentation follows a predictable four-level structure
- Forces discipline about which details belong at which abstraction level
- Requires ongoing training for contributors unfamiliar with C4

ADR-002: Event-Driven Cross-Domain Integration as Default Style

ID	ADR-002
Status	Accepted
Review	Annual review — 2027-Q1

Context

Synchronous REST calls between domain services were creating hidden dependencies, making services difficult to evolve independently and causing cascading failures.

Decision

All integration across bounded context boundaries MUST use asynchronous, typed, versioned domain events published to Kafka. Synchronous calls across domain boundaries are prohibited without an explicit ADR exception.

Alternatives Considered

- Synchronous REST only — simple but tight coupling
- GraphQL federation — good for query but poor for event contracts
- Shared database — explicitly rejected, couples schema evolution

Consequences

- Services can be deployed, scaled, and evolved independently
- Failure in one domain does not cascade synchronously
- Event log provides natural audit trail and replay capability
- Requires event schema governance to prevent incompatible formats

ADR-003: Use LangGraph for Stateful Workflow Execution

ID	ADR-003
Status	Accepted
Review	Annual review — 2027-Q1

Context

Agent orchestration requires stateful multi-step workflows with conditional branching, human-gate suspension/resumption, and cyclical iteration. Linear pipeline frameworks could not support these patterns without significant custom code.

Decision

LangGraph is adopted as the workflow orchestration engine. Its graph-based state machine model maps naturally to procurement workflows. State checkpointing enables workflow suspension and resumption at human decision gates.

Alternatives Considered

- AutoGen — good for conversation but weaker state persistence
- Temporal — strong durability but heavyweight operational curve
- Custom state machine — maximum control but high build cost

Consequences

- Procurement workflows expressed as first-class graph definitions
- Suspension and resumption at human gates is built-in
- LangGraph community and tooling benefits the project
- Requires version pinning — upgrades need regression testing

---

ADR-004: Polyglot Persistence with PostgreSQL as Default Operational Store

ID	ADR-004
Status	Accepted
Review	Annual review — 2027-Q1

Context

Procurement data has radically different access patterns: ACID transactions for POs; vector similarity for contract analysis; graph traversal for supplier networks; immutable append-only audit logs.

Decision

PostgreSQL 15+ is the default operational store. PGVector is co-located for semantic search. Neo4j for relationship-centric queries. ClickHouse for immutable audit and analytics. S3-compatible blob for documents. Redis for caching.

Alternatives Considered

- PostgreSQL only — simple but poor fit for graph and vector workloads
- MongoDB — flexible schema but weaker transactional guarantees
- Distributed NewSQL — strong but higher operational cost

Consequences

- Each store is optimally matched to its workload
- PGVector co-location avoids separate vector DB operational overhead
- Higher operational complexity — teams must maintain multiple storage technologies

---

ADR-005: Universal Contract Aggregate as Domain-Neutral Abstraction

ID	ADR-005
Status	Accepted
Review	Annual review — 2027-Q1

Context

Organisations manage multiple contract types with overlapping attributes. Separate models would fragment the codebase and make cross-contract analytics impossible.

Decision

A single Universal Contract aggregate parameterised by contract type is defined. All contract attributes, obligations, documents, risk profiles, and lifecycle state are owned by this aggregate.

Alternatives Considered

- Separate entity per type — simple but fragmented analytics
- Contract template approach — flexible but poor for invariant enforcement
- External CLM via API — avoids building but sacrifices deep platform integration

Consequences

- Single audit trail for all contract types
- Cross-contract analytics possible with uniform model
- AI analysis pipelines work on a single schema
- Terms map pattern requires disciplined schema governance

ADR-006: Plugin Manifests with SemVer and Explicit Capability Scopes

ID	ADR-006
Status	Accepted
Review	Annual review — 2027-Q1

Context

Platform operators need a reliable way to understand what capabilities a plugin requires, what data it accesses, and whether it is compatible with a given platform version.

Decision

Every plugin MUST declare a manifest with SemVer version, explicit capability tokens, platform compatibility range, and dependency declarations. The Marketplace rejects any submission that does not include a valid signed manifest.

Alternatives Considered

- No formal manifest — simple but impossible to enforce capabilities
- Custom DSL — expressive but high contributor barrier
- Kubernetes RBAC-style binding — familiar but not portable across deployment modes

Consequences

- Platform can enforce declared capabilities at runtime via OPA
- Compatibility matrix enables automated upgrade safety checks
- Publishers have a clear contract surface to design against

ADR-007: WebAssembly-First Sandboxing with Container Fallback

ID	ADR-007
Status	Accepted
Review	Annual review — 2027-Q1

Context

Plugin execution requires strong isolation. Container isolation provides good boundaries but has non-trivial startup latency. WebAssembly runtimes offer sub-millisecond startup with deterministic resource limits.

Decision

Plugin sandbox defaults to WebAssembly (WASI-compliant runtime). Plugins that cannot be compiled to Wasm may use container sandboxes with explicit manifest justification. Wasm hard limits: 256MB memory, 10-second CPU, no host filesystem, declared egress URLs only.

Alternatives Considered

- Container-only — universally compatible but higher resource overhead
- Process isolation — simple but weak blast radius containment
- No sandbox — impossible to run untrusted code safely

Consequences

- Sub-millisecond sandbox startup for Wasm-compiled plugins
- Hard resource limits enforced by runtime — not by policy
- Binary portability across all deployment environments
- Requires Wasm compilation target — limits language choice for some plugin types

---

ADR-008: OPA-Based ABAC for Fine-Grained Authorisation

ID	ADR-008
Status	Accepted
Review	Annual review — 2027-Q1

Context

OpenS2P requires authorisation decisions that depend on multiple attributes simultaneously — user role, organisation, spend band, category, document classification, and time. Simple RBAC cannot express these conditions. Platform needs policy-as-code that is version-controlled and independently auditable.

Decision

Open Policy Agent (OPA) with Rego-language policies is adopted for all authorisation decisions. Policies are stored in a dedicated repository, version-controlled alongside platform code, deployed as OPA sidecars, and every evaluation is logged to ClickHouse.

Alternatives Considered

- Casbin — mature RBAC/ABAC but embedded in code rather than externally auditable

- Custom middleware — full control but no standardised audit
- AWS Cedar — expressive but creates cloud provider dependency

Consequences

- All authorisation decisions are externally auditable and testable
- Policy changes do not require code deployment
- OPA test tooling integrates with CI pipeline
- Rego has a learning curve for contributors unfamiliar with logic programming

---

ADR-009: ClickHouse for Immutable Audit, Trace Analytics, and Policy Evidence

ID	ADR-009
Status	Accepted
Review	Annual review — 2027-Q1

Context

Procurement platforms must maintain tamper-evident audit trails for regulatory compliance. Traditional RDBMS audit tables are mutable and can be modified by database administrators.

Decision

ClickHouse is adopted as the immutable audit and analytics store. All domain events, agent decisions, and policy evaluations are written as append-only records. No DELETE or UPDATE API. Storage immutability enforced at infrastructure level.

Alternatives Considered

- PostgreSQL audit tables — familiar but mutable
- Elasticsearch — good search but complex immutability enforcement
- Dedicated audit SaaS — strong but creates external dependency for air-gapped deployments

Consequences

- Tamper-evident audit trail satisfying financial and regulatory requirements
- Columnar storage enables fast aggregate analytics over millions of events
- AI agent decisions queryable alongside business events
- Separate operational overhead for ClickHouse cluster management

---

ADR-010: vLLM for Self-Hosted Enterprise Inference Serving

ID	ADR-010
Status	Accepted
Review	Annual review — 2027-Q1

Context

Enterprise and regulated deployments cannot send sensitive procurement data to external LLM APIs. They require local, air-gapped inference with dynamic batching for concurrent agent execution.

Decision

vLLM is adopted as the reference inference server for self-hosted deployments. It provides PagedAttention-based memory management, continuous batching, and support for major open-weight model families. The platform's LLM abstraction layer routes to vLLM in enterprise mode and external APIs in SaaS mode.

Alternatives Considered

- Ollama — excellent for developer laptops but limited production throughput
- TGI — comparable performance but less active community trajectory
- Direct model loading — requires significant custom serving infrastructure

Consequences

- Air-gapped inference capability for regulated enterprise deployments
- Consistent LLM abstraction layer — application code unchanged between SaaS and enterprise modes
- GPU cluster required — significant infrastructure cost
- Model selection and quantisation requires ML expertise on customer operations team

---

ADR-011: Separate Marketplace Trust Services from Plugin Execution Services

ID	ADR-011
Status	Accepted
Review	Annual review — 2027-Q1

Context

The Marketplace control plane (publication, signing, moderation) and Plugin Execution runtime (sandbox invocation, capability enforcement) have different trust requirements and threat models.

Decision

Marketplace Trust Services and Plugin Execution Services are deployed as separate services with a defined API boundary. Trust Service is the authority on what is approved. Execution Service is the authority on how plugins run. Neither can bypass the other.

Alternatives Considered

- Monolithic plugin service — simpler but couples publication risk with execution risk
- Trust as sidecar to execution — reduces overhead but tightens coupling

Consequences

- A compromised execution runtime cannot affect the trust chain or approve new plugins
- Each service can be scaled and audited independently
- API boundary between services must be kept stable — changes require compatibility planning

---

ADR-012: OpenTelemetry as Mandatory Observability Standard

ID	ADR-012
Status	Accepted
Review	Annual review — 2027-Q1

Context

OpenS2P runs across multiple deployment environments with different infrastructure stacks. Proprietary observability instrumentation locks the platform to specific backend tooling and makes it impossible to offer portability across customer environments.

Decision

All OpenS2P services MUST instrument using the OpenTelemetry SDK from the first commit. OTEL is the only supported instrumentation API. OTEL Collector is the mandatory signal aggregation component in all deployment topologies.

Alternatives Considered

- Prometheus-only — good ecosystem but traces require separate instrumentation
- Datadog agent — excellent tooling but vendor lock-in and SaaS-only model
- Custom structured logging only — lowest cost but no distributed tracing

Consequences

- Single instrumentation investment works across all deployment targets
- OTEL Collector routing enables backend-agnostic deployment
- Growing contributor ecosystem means libraries are increasingly pre-instrumented
- OTEL SDK maturity varies by language

ADR-013: Contract-First APIs with OpenAPI and AsyncAPI Artifacts

ID	ADR-013
Status	Accepted
Review	Annual review — 2027-Q1

Context

Without contract-first API discipline, implementations drift from intended interfaces, integration partners build against unstable APIs, and breaking changes occur without notice.

Decision

All REST APIs MUST have a committed OpenAPI 3.1 specification before implementation. All event schemas MUST have a committed AsyncAPI 3.x specification before the first event is published. Automated contract tests run in CI to enforce conformance.

Alternatives Considered

- Implementation-first with auto-generated docs — fast start but poor stability guarantees

- GraphQL schema-first — good for query APIs but poor fit for event contracts

Consequences

- Integration partners can code against stable, published contracts
- Breaking changes are impossible without a visible specification change
- CI contract failures surface incompatibilities before integration environments
- Requires team discipline to maintain specs before implementation

---

ADR-014: Compatibility Windows for Plugins, Events, and APIs

ID	ADR-014
Status	Accepted
Review	Annual review — 2027-Q1

Context

As the platform evolves, published plugins, connectors, and event consumers must continue to work without requiring simultaneous upgrades across all ecosystem participants.

Decision

Plugin APIs: 12-month deprecation notice for breaking changes. Event schemas: 6-month dual-publish period. REST APIs: 2 major versions supported simultaneously. Connector manifests: major version bump required with migration tooling.

Alternatives Considered

- No compatibility guarantees — maximum velocity but destroys ecosystem trust
- LTS model — strong ecosystem signal but slows platform evolution

Consequences

- Marketplace publishers can plan SDK upgrades against known timelines
- Enterprise adopters can plan upgrade cycles without emergency patches
- Platform team must maintain compatibility test suites against prior versions
- Dual-publish period for events increases operational complexity

---

ADR-015: Schema-Per-Tenant PostgreSQL Partitioning as Default Isolation Model

ID	ADR-015
Status	Accepted
Review	Annual review — 2027-Q1

Context

Multi-tenancy requires strong data isolation. Three models exist: shared tables, schema-per-tenant, database-per-tenant. The choice significantly affects isolation strength, operational overhead, and scalability ceiling.

Decision

Schema-per-tenant is the default isolation model. Each tenant receives a dedicated PostgreSQL schema. Data access layer enforces schema routing at connection time. Row-level security provides defence-in-depth. Enterprise tier optionally supports database-per-tenant.

Alternatives Considered

- Shared tables with tenant_id — simple but weakest isolation; missing WHERE clause leaks cross-tenant data
- Database-per-tenant — strongest isolation but prohibitive operational overhead at scale

Consequences

- Strong data isolation without the operational cost of separate clusters
- Schema-level grants provide additional isolation enforcement beyond application logic
- Performance isolation not guaranteed — a noisy tenant can affect shared infrastructure
- Schema proliferation at very high tenant counts requires connection pooler tuning

---

ADR-016: Define and Enforce AI Quality SLOs as Platform-Level Contracts

ID	ADR-016
Status	Accepted
Review	Annual review — 2027-Q1

Context

AI quality metrics (hallucination rate, citation coverage, tool success rate, workflow completion) were tracked informally per-team with no platform-level standards, making it impossible to compare agent quality across domains or enforce quality gates in CI.

Decision

AI quality metrics are promoted to platform-level SLOs. Every agent must pass the standard evaluation matrix (Chapter 13) before production deployment. Production metrics are monitored continuously in Grafana with alerting. SLO degradation triggers automatic agent version rollback.

Alternatives Considered

- Informal per-team metrics — lowest overhead but no platform-level quality baseline
- External AI evaluation service — comprehensive but creates dependency and data sharing risk for enterprise

Consequences

- AI quality is now a measurable, enforceable platform contract — not a subjective assessment
- Evaluation results are stored in ClickHouse alongside business events — fully auditable
- CI gate prevents deployment of prompts that do not meet quality standards
- Evaluation dataset must be maintained and expanded as agent capabilities grow

---

24. Technology Radar

The OpenS2P Technology Radar classifies technologies into five rings: Adopt (production default), Trial (use on projects to learn), Assess (proof-of-concept only), Hold (proceed with caution — not recommended for new work), and Retire (being phased out). The radar is reviewed quarterly by the TSC.

ADOPT	TRIAL	ASSESS	HOLD	RETIRE
PostgreSQL 15+	LangGraph	Wasm Component Model	Shared DB cross-domain	ZooKeeper (Kafka < 3.x)
PGVector	ClickHouse	Cedar (policy language)	Sync REST across bounded contexts	Python 3.8 (EOL)
Kubernetes	vLLM	Ollama (production scale)	Hardcoded secrets in env	OpenAPI 2.x (Swagger)
OpenTelemetry	Neo4j	OpenFGA	Proprietary observability SDK	JWT long-lived tokens (> 1h)
OPA	Wasm (WASI)	Temporal	Plugin direct internal API	
Keycloak	AsyncAPI 3.x	Lancedb		
HashiCorp Vault	Grafana Tempo	SurrealDB		
Kafka	Argo Rollouts	Deno (plugin alt)		
Redis	Pact (contract testing)			
TypeScript				
React				
Docker				

Figure 39: OpenS2P Technology Radar — Q3 2026 (5 rings including Retire)

25. Release Architecture, Versioning & Deprecation

25.1 Versioning Policy

Release Type	Frequency	Compatibility Guarantee	Support Window
Patch	As needed (security: 24h target)	Fully backward compatible	Always upgrade
Minor	Monthly	Backward compatible — new capabilities only	12 months or 2 minor versions

Major	Annually (target)	Breaking changes with migration guide	24 months from GA
LTS	Every other major	Extended support SLA	36 months from GA

25.2 Roadmap: v3.0 → v5.0

Phase	Version	Key Deliverables	Status
Foundation	v3.0	Core C4 model, event backbone, basic agent lifecycle, PostgreSQL + PGVector	Complete
Formalisation	v4.0	Full ADR catalog, formal domain model, security architecture, deployment topology	Complete
Flagship	v4.1	Complete governance model, marketplace architecture, enterprise topology	Complete
Expanded	v4.2	This document — AI architecture, testing architecture, compliance, evaluation framework	Approved
Enterprise GA	v5.0	Multi-tenancy GA, certified marketplace, vLLM GA, enterprise SLOs, ISO 27001 alignment	Planned Q2 2027
Ecosystem	v5.x	Partner plugin program, ISV certification, multi-region SaaS, AI Act compliance	Planned 2027–2028

Appendix A: Complete Diagram Index

Figure	Diagram Title	Chapter	Type
1	Business Capability Hierarchy	2	Capability Model
2	Domain Dependency Map — Event Flow	3	Domain Architecture
3	Binding Architecture Principles	4.1	Principles Diagram
4	C4 Level 1 — System Context	5	C4 Context
5	C4 Level 2 — Container Architecture	6	C4 Container
6	C4 Level 3 — Workflow Orchestrator	7.1	C4 Component
7	C4 Level 3 — Security Services	7.2	C4 Component
8	C4 Level 3 — Marketplace Components	7.3	C4 Component
9	C4 Level 3 — Connector Framework	7.4	C4 Component
10	Core Procurement Ontology	8	Ontology Diagram
11	Bounded Context Map	8.1	Domain Architecture
12	Data Store Responsibility Matrix	9	Data Architecture
13	Data Lineage Event Schema	9.1	Data Architecture
14	AI Reference Architecture	10.1	AI Architecture
15	Multi-Agent Handoff Sequence	10.2	UML Sequence
16	Prompt Assembly Pipeline	11	AI Architecture
17	Three-Layer Memory Architecture	12	AI Architecture

18	Organisational Memory Embedding Pipeline	12.1	AI Architecture
19	AI Evaluation Metrics Matrix	13.1	AI Evaluation
20	AI Evaluation Pipeline Sequence	13.2	UML Sequence
21	Agent Red Team Test Categories	13.3	Security / AI
22	Agent Request Execution Sequence	14	UML Sequence
23	Standardised Agent Lifecycle	15	Process Diagram
24	HITL Gate Configuration Model	15.1	Control Architecture
25	Plugin Runtime — Isolation Architecture	16	Security Architecture
26	Plugin Publication and Installation Sequence	17	UML Sequence
27	Security Reference Architecture — Trust Zones	18	Security Reference
28	Identity Trust Chain Sequence	18.1	UML Sequence
29	SaaS Multi-Tenant Production Topology	19.1	Deployment Topology
30	Enterprise Air-Gapped Topology	19.2	Deployment Topology
31	Testing Architecture — Eight Disciplines	20	Testing Architecture
32	CI/CD Testing Pipeline Gate Structure	20.1	UML Sequence
33	Testing Ownership Matrix	20.2	Governance
34	Compliance Framework Coverage Matrix	21.1	Compliance
35	Audit Evidence Architecture	21.2	Compliance / Security
36	AI Regulatory Alignment	21.3	Compliance / AI

	Matrix		
37	OpenS2P Governance Hierarchy	22	Governance
38	Architecture Proposal Lifecycle	22.1	UML Sequence
39	Technology Radar (5 rings)	24	Technology Radar

Appendix B: Glossary

Term	Definition
ADR	Architecture Decision Record — an immutable record of a significant architectural choice, its context, alternatives, and consequences.
Aggregate Root	The primary entity in a domain aggregate that controls access to all objects within the aggregate boundary.
Bounded Context	A defined boundary within which a particular domain model applies and is consistent. Cross-context integration uses events.
C4 Model	Context, Container, Component, Code — a hierarchical software architecture documentation approach.
Confidence Score	A numeric signal (0.0–1.0) emitted by an AI agent indicating its certainty in its output or recommendation.
Dead Letter Topic	A Kafka topic that receives messages that could not be processed after maximum retry attempts.
Domain Event	An immutable record of something that happened in the business domain. Always past tense. Crosses bounded context boundaries.
ECE	Expected Calibration Error — a statistical measure of how well a model's confidence scores correspond to actual correctness rates.
Hallucination	An AI output that contains factually incorrect statements not supported by retrieved context or ground truth.
HITL	Human-in-the-Loop — a mandatory human decision gate in an AI workflow, triggered when agent

	confidence falls below the configured threshold.
LangGraph	A graph-based workflow orchestration library for stateful, multi-step AI agent chains.
LLM	Large Language Model — a transformer-based AI model capable of generating and reasoning over natural language.
OPA	Open Policy Agent — a general-purpose policy engine that evaluates Rego-language policies for authorisation decisions.
Organisational Memory	The persistent, tenant-wide knowledge base stored in PGVector, Neo4j, and PostgreSQL, queried by the retrieval layer during every agent invocation.
Plugin	A signed, sandboxed extension that adds capabilities to the OpenS2P platform via the defined extension API.
Prompt Registry	The versioned catalogue of all prompts used by OpenS2P agents. New prompt versions require evaluation regression before production deployment.
RAG	Retrieval-Augmented Generation — an AI pattern where the LLM is provided with retrieved relevant context before generating a response.
RFC	Request for Comments — the formal mechanism for proposing architectural changes to OpenS2P.
SemVer	Semantic Versioning. Version format MAJOR.MINOR.PATCH with defined compatibility semantics.
TSC	Technical Steering Committee — the highest authority in OpenS2P architecture governance.
Universal Contract	The domain-neutral aggregate that represents any legal agreement in the OpenS2P platform, parameterised by contract type.
vLLM	An inference server for open-weight LLMs using PagedAttention and continuous batching for efficient GPU utilisation.
Wasm	WebAssembly — a binary instruction format used as the primary sandboxed execution environment for OpenS2P plugins.
Zero Trust	A security model where no component implicitly trusts another. Every request is authenticated and authorised explicitly.

Appendix C: API Surface Summary

Service	API Style	Auth	Version	Spec
Sourcing Service	REST	JWT + OPA	v1.0	/api/sourcing/openapi.yaml
Contract Service	REST + Events	JWT + OPA	v1.0	/api/contract/openapi.yaml
Supplier Service	REST + Events	JWT + OPA	v1.0	/api/supplier/openapi.yaml
Purchasing Service	REST + Events	JWT + OPA	v1.0	/api/purchasing/openapi.yaml
Invoice Service	REST + Events	JWT + OPA	v1.0	/api/invoice/openapi.yaml
Workflow Orchestrator	gRPC + REST	mTLS + JWT	v1.0	/api/workflow/openapi.yaml
Plugin Registry	REST	JWT + API Key	v1.0	/api/marketplace/openapi.yaml
Identity Service	OIDC / OAuth 2.0	Client credentials	v1.0	/.well-known/openid-configuration
Policy API	REST	mTLS (internal)	v1.0	/api/policy/openapi.yaml
Audit Query API	REST (read-only)	JWT + OPA	v1.0	/api/audit/openapi.yaml
Domain Events (Kafka)	AsyncAPI	SASL/SCRAM	v1.0	/api/events/asyncapi.yaml
AI Evaluation API	REST	JWT + OPA	v1.0	/api/evaluation/openapi.yaml

Appendix D: Architecture Review Checklist

Every feature, service, or significant change to OpenS2P MUST be assessed against this checklist before the implementation PR is opened. The checklist is completed by the implementing engineer and reviewed by the domain module owner. Items marked MUST are blocking — the PR MUST NOT be merged until all MUST items are satisfied.

#	Category	Checklist Item	RFC 2119	Reviewer
1	API Design	REST API documented in OpenAPI 3.1 spec committed before code	MUST	Module Owner
2	API Design	All request/response schemas include examples	MUST	Module Owner
3	API Design	Breaking changes flagged with deprecation notice and migration path	MUST	Domain Owner
4	Events	All domain events documented in AsyncAPI 3.x spec	MUST	Module Owner
5	Events	Event schema backward-compatible or dual-publish period declared	MUST	Domain Owner
6	Security	STRIDE threat model completed for new attack surface	MUST	Security Reviewer
7	Security	OPA policy written and tested for new endpoints	MUST	Security Reviewer
8	Security	No secrets, tokens, or PII in code, config, or logs	MUST	CI Gate (automated)
9	AI	Prompt version registered in Prompt Registry	MUST (AI changes)	AI Platform Team
10	AI	Evaluation suite updated and passing (hallucination, citation, tool success)	MUST (AI changes)	AI Platform Team
11	AI	PII redaction verified for all new prompt context	MUST (AI changes)	AI Platform Team

		fields		
12	Testing	Unit test coverage ≥ 90% on domain layer	MUST	Module Owner
13	Testing	Integration tests cover all new repository methods	MUST	Module Owner
14	Testing	Pact consumer contract tests updated for API changes	MUST (API changes)	Consumer Teams
15	Observability	OTEL spans added for all new processing steps	MUST	Module Owner
16	Observability	Metrics added for new throughput and error rate signals	SHOULD	Module Owner
17	Architecture	ADR written if decision affects platform-wide patterns	MUST (cross-cutting)	Core Maintainer
18	Architecture	C4 diagram updated if component structure changed	SHOULD	Module Owner
19	Multi-tenancy	Tenant isolation verified — no cross-tenant data access possible	MUST	Security Reviewer
20	Compliance	Data retention and audit event coverage confirmed	MUST	Risk & Compliance Team
21	Deployment	Helm chart updated; deployment topology documented	SHOULD	Platform Team
22	Documentation	README and CHANGELOG updated	SHOULD	Implementing Engineer

Figure L: Architecture Review Checklist — Pre-Implementation Gate

Appendix E: Document Version History

Version	Date	Status	Summary of Changes	Author
v3.0	Q1 2025	Superseded	Initial conceptual architecture. C4 Level 1–2, basic agent lifecycle, PostgreSQL + PGVector baseline.	OpenS2P Core Team
v4.0	Q2 2025	Superseded	Full ADR catalog (ADR-001–ADR-012), formal domain model, security reference architecture, deployment topology.	OpenS2P Core Team
v4.1	Q3 2025	Superseded	Complete governance model, marketplace architecture, enterprise air-gapped topology, ADR-013–ADR-015, 37-diagram index.	OpenS2P Core Team
v4.2	Q2 2026	Superseded	Business Capability Model, Domain Dependency Map, consolidated AI Architecture, Prompt Architecture, Memory Architecture, AI Evaluation Framework (ADR-016), Testing Architecture (8 disciplines), Compliance Architecture (8 frameworks), 5-ring Technology Radar.	OpenS2P Core Team
v4.3	Q1 2027	Current —	Architecture Goals	OpenS2P Core

		Approved	(G-01–G-10), Architecture Constraints (C-01–C-12), Principle Traceability Matrix, Architecture Decision Summary (The Why), Domain Ownership Matrix, 3× Deployment Profiles, Capacity Planning, AI Cost Governance, Disaster Recovery Strategy, Architecture Review Checklist, Version History.	Team
v4.4	Q1 2027	Current — Approved — FINAL	DDD Context Map (§3.3), Integration Pattern Catalog (§18.3) with 7 patterns, Data Classification Matrix (§9.2), CI/CD Reference Architecture with supply chain security (§20.3–20.4), How to Read guide, Repository Layout (Appendix H), Architecture Maturity Model (Appendix I). Document declared feature-complete.	OpenS2P Core Team
v4.4	Q1 2027	Current — Approved — FINAL	DDD Context Map (§3.3), Integration Pattern Catalog (§18.3 — 7 patterns), Data Classification Matrix (§9.2), CI/CD Reference Architecture with supply chain security (§20.3–20.4), How to Read guide, Repository	OpenS2P Core Team

			Layout (Appendix H), Architecture Maturity Model (Appendix I). Document declared feature-complete.	

Figure M: Document Version History

Appendix F: Technical Debt Register

The Technical Debt Register records known architectural debts — intentional shortcuts, deferred decisions, or areas where current implementation diverges from the target architecture. Every item MUST have an owner and a target resolution version. Unresolved items are reviewed at every major release.

ID	Description	Impact	Owner	Target Version	Status
TD-001	Connector Framework does not yet enforce egress URL allowlist at runtime for HTTP connectors. Declared in manifest but not verified during execution.	Security — medium. Connector could call undeclared endpoints.	Platform Team	v5.0	Open
TD-002	Knowledge Context embedding refresh is triggered by full document re-index, not incremental chunk updates. Large documents cause unnecessary re-embedding cost.	Cost / Performance — medium.	AI Platform Team	v5.0	Open
TD-003	Plugin Wasm sandbox CPU	Resource isolation — low	Platform Team	v5.1	Open

	time enforcement uses wall clock, not CPU time. Under high host load, a plugin may consume more CPU than declared limit.	risk for now.			
TD-004	ClickHouse audit store does not yet have automated evidence pack generation for ISO 27001 or SOC 2 templates. Manual export currently required.	Compliance — medium. Increases audit cost.	Risk & Compliance Team	v5.0	Open
TD-005	Prompt Registry does not yet enforce evaluation pass before production promotion. Evaluation is run in CI but human override is still possible without TSC review.	AI Quality Governance — medium.	AI Platform Team	v5.0	In Progress
TD-006	Kafka topic ACLs are currently set at cluster level, not per-topic per-tenant. A service with cluster READ access can consume topics it should not.	Security — medium. Requires Kafka ACL refactor.	Platform Team	v5.0	Planned
TD-007	OPA policy bundle refresh interval is 30 seconds. There is a 30-second	Security — low. Policy revocation not yet real-time.	Security Team	v5.1	Open

	window where a newly revoked permission remains valid after policy update.				

Figure N: Technical Debt Register

Appendix G: Architecture Risk Register

The Architecture Risk Register records risks to the platform architecture that are not yet fully mitigated. Unlike technical debt (known implementation gaps), architecture risks are uncertainties that could require design changes if they materialise. Every risk has an owner, likelihood/impact rating, and mitigation strategy.

ID	Risk Description	Likelihood	Impact	Mitigation Strategy	Owner
AR-001	LangGraph API breaks compatibility in a major version update, requiring significant orchestration rewrites.	Medium	High	Version pin LangGraph in all services. Maintain orchestration abstraction layer. Evaluate alternative frameworks quarterly (see Technology Radar: Temporal in Assess).	AI Platform Team
AR-002	GPU availability constraints during peak procurement periods (quarter-end, year-end) cause agent queue buildup beyond acceptable latency SLOs.	High	High	GPU auto-scaling node pool. Batch vs interactive latency class routing. External LLM API fallback for non-sensitive workflows.	SRE Team
AR-003	Wasm runtime (wasmtime/wasmer) vulnerability	Low	High	CVE monitoring on Wasm runtime versions. Forced	Platform Team

	discovered post-plugin publication, requiring emergency sandbox patching across all installed plugins.			plugin sandbox restart on security patches. Plugin capability guard as defence-in-depth.	
AR-004	Schema-per-tenant model hits PostgreSQL schema count limits at very high tenant counts (10,000+ schemas), causing connection pooler and DDL management issues.	Low (near-term)	Medium	Connection pooler tuning (PgBouncer). Schema consolidation strategy for inactive tenants. Database-per-cluster migration path for large tenants.	Platform Team
AR-005	EU AI Act high-risk classification for AI-assisted contract signing or supplier approval decisions triggers compliance obligations not yet fully addressed.	Medium	High	Legal review of use cases against EU AI Act risk tiers. HITL gates already in place. Transparency logging already in ClickHouse. Formal risk assessment for v5.0.	Risk & Compliance Team
AR-006	Kafka becomes an operational bottleneck as event volume grows: complex topic management, consumer group lag, and cross-region MirrorMaker 2 latency.	Medium	Medium	Topic compaction and retention tuning. Consumer lag monitoring with auto-alerting. Evaluate Kafka replacements in Technology Radar Assess ring for v6.x.	SRE Team

Figure O: Architecture Risk Register

Appendix H: Architecture Repository Layout

The following is the reference monorepo structure for OpenS2P. Contributors MUST follow this structure. New bounded contexts, services, plugins, and connectors are created by replicating the corresponding template directory. A CI directory-structure lint check enforces compliance.

OpenS2P Monorepo Reference Structure	
docs/	— Architecture and design documentation root
adr/	— Architecture Decision Records (ADR-NNN-slug.md per ADR)
rfc/	— Requests for Comments (RFC-NNN-slug.md per RFC)
architecture/	— Architecture Vision document and diagrams source files
api/	— OpenAPI 3.1 specifications (one YAML per service)
asyncapi/	— AsyncAPI 3.x specifications (one YAML per domain event group)
diagrams/	— Structurizr DSL / draw.io source files for all diagrams
services/	— Core platform services (one directory per bounded context)
{context}/ marketplace/	— e.g. supplier/, contract/, purchasing/, invoice/, payment/, risk/, knowledge/,
src/domain/	— Entities, aggregates, value objects, domain events (no infrastructure imports)
src/application/	— Use cases, command handlers, query handlers
src/infrastructure/	— Repository implementations, Kafka adapters, external service clients
src/presentation/	— REST controllers, DTOs, request/response mappers
tests/unit/	— Domain layer unit tests (no DB, no network)
tests/integration/	— Infrastructure tests (Testcontainers)
tests/contract/	— Pact consumer/provider contract tests
openapi.yaml	— OpenAPI 3.1 spec (committed before implementation — C-07)
asyncapi.yaml	— AsyncAPI 3.x spec for events published by this service (C-08)
Dockerfile	— Multi-stage distroless build

helm/	— Helm chart for this service
plugins/	— First-party plugin packages
{plugin-name}/	— manifest.json + src/ + schemas/ + tests/
connectors/	— First-party connector packages (same structure as plugins)
sdk/	— Official OpenS2P SDK packages
typescript/	— TypeScript SDK (@opens2p/sdk)
python/	— Python SDK (opens2p-sdk)
infrastructure/	— Infrastructure-as-code
helm/	— Platform-wide Helm umbrella chart
terraform/	— Optional cloud infrastructure (cloud-specific, not in core)
kubernetes/	— Raw Kubernetes manifests for air-gapped deployments
.github/workflows/	— ci.yml (PR gate) · release.yml (publish) · security.yml (daily scan)
GOVERNANCE.md	— TSC composition, RFC/ADR process, breaking change authority matrix
CONTRIBUTING.md	— Contributor guide, coding standards, local dev setup, PR checklist
CHANGELOG.md	— Semantic versioned change history (Conventional Commits format)

Figure U: OpenS2P Monorepo Reference Structure

Appendix I: Architecture Maturity Model

The OpenS2P Architecture Maturity Model enables adopting organisations to assess their current state and identify capabilities required to advance. It is also a platform roadmap planning tool — each major version targets a maturity level milestone.

Level	Name	Description	Key Capabilities Required	Platform Version	Typical Adopter
1 — Pilot	Pilot	Core platform running. Basic procurement workflows	PR/PO lifecycle · Supplier onboarding · Basic contract	v4.x	Innovation team / PoC

		operational. Single tenant. Manual approvals.	storage · External LLM API · Single K8s cluster · Starter profile		
2 — Production	Production	Full S2P lifecycle operational. HA deployment. Multi-tenant. AI-assisted approvals in selected workflows.	Three-way match · Invoice automation · Agent-assisted sourcing · HA PostgreSQL + Kafka · OPA enforced · Audit trail active	v5.0	Single enterprise, departmental rollout
3 — Enterprise	Enterprise	Organisation-wide deployment. Air-gap option. Full compliance controls. ERP and supplier network connectors live.	ERP connector live · Self-hosted vLLM · SOX audit evidence automated · Multi-region option · Security pen-tested · ISO 27001 aligned	v5.x	Large enterprise, global rollout
4 — AI Native	AI Native	AI agents operating autonomously across full S2P lifecycle. Confidence scoring and HITL gates calibrated. Org memory active.	All 9-stage agents deployed · Memory architecture active · Prompt registry versioned · AI eval SLOs met · AI cost governance enforced · Red team running	v6.0	AI-forward enterprise, COE model
5 — Ecosystem	Marketplace Ecosystem	Active plugin/connector marketplace. ISV partners publishing certified plugins. Community-driven capability extension.	Certified plugin program · ISV integrations · Plugin trust tiers operating · Community RFC pipeline · SDK ecosystem · Multi-org governance	v6.x+	Platform-as-ecosystem / OSS foundation

Figure V: OpenS2P Architecture Maturity Model — Five Levels

Maturity Advancement Gates

- Level 1 → 2: Three-way match operational, HA data stores deployed, OPA policies enforced, audit trail active.
- Level 2 → 3: ERP connector live and integration-tested, self-hosted vLLM operational, compliance evidence auto-generated, pen test passed.
- Level 3 → 4: All agents deployed and meeting AI evaluation SLOs, organisational memory indexed, HITL thresholds calibrated, AI cost governance enforced.
- Level 4 → 5: At least 5 certified partner plugins in marketplace, ISV developer program open, community contributions accepted via RFC process.

